

Automatic Generation of Formulae for Decidable Fragments of First-Order Logic

Manuele Borgia

Anno Accademico 2025-2026

Contents

Introduction	4
1 First-Order Logic	6
1.1 Syntax	6
1.1.1 Formulae as Parse Trees	7
1.2 Semantics	8
1.2.1 Concrete Evaluation over a Finite Domain	9
1.3 Decidability and Undecidability	9
1.3.1 Semi-Decidability	10
2 Decidable Fragments	11
2.1 The Two-Variable Fragment	11
2.1.1 Tree Representation and Variable Recycling	12
2.1.2 Finite Model Property and SAT Generation	13
2.1.3 Complexity	13
2.2 The Fluted Fragment	14
2.2.1 Tree Representation and the Scope Stack	14
2.2.2 Complexity	15
2.3 The Guarded Fragment	16
2.3.1 Tree Representation and Guard Generation	16
2.3.2 Complexity	17
2.4 The Unary Negation Fragment	18
2.4.1 Tree Representation and Variable Dependencies	18
2.4.2 Complexity	20
2.5 Propositional Modal Logic and the Standard Translation	20
2.5.1 The Standard Translation	20
2.5.2 Tree Representation and Variable Alternation	21
2.5.3 Complexity	21
2.5.4 Summary	23
3 System Architecture	24
3.1 Lexical Abstraction	24
3.2 Formula Generation	26
3.3 The FO2 SAT Model and Visitor	28

3.4	External Verification: The Vampire Actor	29
3.5	Architecture Overview	31
3.6	System Interface and CLI	33
3.6.1	Application Controller	33
3.6.2	Self-Documenting Interface	33
4	Implementation	35
4.1	Development Environment	35
4.2	The Abstract Syntax Tree (AST)	36
4.2.1	Node Hierarchy and Interfaces	36
4.2.2	The Negation Normal Form (NNF)	36
4.3	The Probabilistic Budget Model	37
4.3.1	Constraint Management and Backtracking	37
4.3.2	Applied Probability Distributions	38
4.4	Randomized Formula Generation	38
4.5	UNSAT Generation Algorithm	42
4.6	Fragment-Specific Methods	43
4.6.1	The Two-Variable Fragment (FO^2) and SAT Construction	43
4.6.2	The Fluted Fragment (FL): Forced Quantification	46
4.6.3	The Guarded Fragment (GF): Guard Generation	47
4.6.4	The Unary Negation Fragment (UNFO): Truncation	48
4.7	Output Serialization and TPTP Formatting	49
4.7.1	Polymorphic Translation and Lexical Sanitization	49
4.7.2	Semantic Roles	49
5	Vampire Theorem Prover	51
5.1	First-Order Clausification	51
5.2	The Superposition Calculus	51
5.3	Saturation Loop and Given Clause Algorithm	52
5.4	Portfolio Solving and CASC Mode	52
5.5	Satisfiability and Finite Model Finding	53
6	Testing and Experimental Validation	54
6.1	The Benchmarking Framework	54
6.2	Execution and Timeout Calibration	54
6.3	Dataset Generation and Analysis	55
6.3.1	FO^2 Fragment Datasets	55
6.3.2	Fluted Fragment Datasets	56
6.3.3	Guarded Fragment Datasets	57
6.3.4	Unary Negation Fragment Datasets	59
6.3.5	Modal Fragment Datasets	60
6.4	Resolution Times and Complexity Analysis	61
6.5	Future Research Directions	65

Introduction

The automation of logical reasoning has been a central challenge in computer science since its earliest days. First-Order Logic (FOL), formalized in the foundations established by Frege’s *Begriffsschrift* and later developed by Russell and Whitehead in *Principia Mathematica* [16], provides a language expressive enough to capture mathematical reasoning, while maintaining the structural precision required to admit computational treatment. Its applications span knowledge representation, formal verification, and artificial intelligence, where the ability to reason automatically over logical formulae has practical consequences ranging from software correctness to ontology reasoning.

While highly expressive, FOL is fundamentally constrained by the undecidability of its *satisfiability problem*, a theoretical limit proven by Church and Turing in the 1930s. This limitation has motivated two complementary lines of research. The first is the development of Automated Theorem Provers (ATPs) — systems such as Vampire [10], which employ saturation-based calculi and sophisticated heuristics to handle large classes of problems efficiently, operating within the theoretical bounds of semi-decidability. The second is the identification of *decidable fragments* of FOL: syntactically restricted subsets that retain sufficient expressive power for practical applications while admitting algorithms that always terminate with a correct answer.

This research analyzes four of the most studied decidable fragments: the Two-Variable Fragment (FO^2), the Guarded Fragment (GF), the Fluted Fragment (FF), and the Unary Negation Fragment (UNF). Each imposes distinct syntactic constraints — on the number of variables, the structure of quantification, or the use of negation — and each captures different classes of reasoning problems that arise naturally in areas such as description logics, database theory, and computational linguistics.

The practical evaluation of decision procedures for these fragments requires extensive testing on formulae that faithfully represent the syntactic diversity of each class. This need motivates the central contribution of this thesis: the design and implementation of **parametric random formula generators** for decidable fragments of FOL. Rather than relying on existing benchmark libraries, which are often sparse or biased toward specific problem structures, our generators produce formulae that are syntactically correct by construction, with configurable structural parameters controlling depth, predicate vocabulary, and quantifier density. A probabilistic *budget model* governs the generation pro-

cess, enabling systematic control over formula complexity. Generated formulae are validated using Vampire, ensuring that satisfiability labels are reliable. For the Two-Variable Fragment, a dedicated finite model construction provides an alternative, model-theoretic approach to generating satisfiable instances.

This thesis is organized as follows. Chapter 1 introduces the theoretical foundations of First-Order Logic. Chapter 2 presents the decidable fragments addressed by this work. Chapter 4 details the implementation and the probabilistic budget model of the formula generator. Finally, Chapter ?? discusses the validation process using Vampire.

Chapter 1

First-Order Logic

This chapter establishes the theoretical foundations upon which this thesis is built. We present a concise yet rigorous overview of First-Order Logic (FOL), detailing its syntax and semantics, and concluding with the fundamental undecidability of its satisfiability problem. This theoretical limit motivates the exploration of decidable fragments, which constitutes the core focus of this work.

1.1 Syntax

The syntax of First-Order Logic strictly defines the rules for constructing valid mathematical expressions. It is built upon a specific alphabet known as a signature [4].

A **signature** (or vocabulary) σ consists of a set of predicate symbols, each associated with a fixed *arity* $n \geq 0$, a set of function symbols, each with a fixed arity $n \geq 0$ (where arity-0 function symbols are conventionally called *constants*), and a countably infinite set of *variables* $\mathcal{V} = \{x_1, x_2, x_3, \dots\}$.

Definition 1.1 (Terms). *The set of terms over a signature σ is defined inductively as follows:*

- every variable $x \in \mathcal{V}$ is a term;
- if f is a function symbol of arity n and $t_1, \dots, t_n$ are terms, then $f(t_1, \dots, t_n)$ is a term.

A term containing no variables is referred to as a ground term.

Definition 1.2 (Formulae). *The set of first-order formulae over σ is defined inductively:*

- **Atomic formulae.** If P is a predicate symbol of arity n and $t_1, \dots, t_n$ are terms, then $P(t_1, \dots, t_n)$ is an atom. Additionally, the expression $t_1 = t_2$ is an equality atom.

- **Boolean connectives.** If φ and ψ are formulae, then so are $\neg\varphi$, $(\varphi \wedge \psi)$, $(\varphi \vee \psi)$, and $(\varphi \rightarrow \psi)$.
- **Quantifiers.** If φ is a formula and $x \in \mathcal{V}$, then $\exists x \varphi$ and $\forall x \varphi$ are formulae.

An occurrence of a variable x in a formula φ is considered *bound* if it lies within the scope of a quantifier ($\exists x$ or $\forall x$); otherwise, it is *free*. A formula containing no free variables is called a *sentence*.

Definition 1.3 (Negation Normal Form). *A formula is in Negation Normal Form (NNF) if the negation operator ($\neg$) is applied exclusively to atomic formulae. Every FOL formula can be transformed into a logically equivalent formula in NNF by systematically pushing negations inward. This is achieved by applying De Morgan’s laws and the duality of quantifiers ($\neg\forall x \varphi \equiv \exists x \neg\varphi$ and $\neg\exists x \varphi \equiv \forall x \neg\varphi$).*

1.1.1 Formulae as Parse Trees

While formulae are typically written as linear strings of text, their inductive definition implies a strict hierarchical structure. In formal logic and computer science, a well-formed formula can be unambiguously represented as a *parse tree* (or abstract syntax tree) [9]. In this tree representation, the internal nodes correspond to logical connectives and quantifiers, while the leaf nodes represent the atomic formulae.

For example, consider a simple academic domain where $S(x)$ denotes “ x is a student”, $C(x)$ denotes “ x is a course”, and $E(x, y)$ denotes “ x is enrolled in y ”. The statement “*Every student is enrolled in at least one course*” can be formalized as $\forall x(S(x) \rightarrow \exists y(C(y) \wedge E(x, y)))$. The tree structure underlying this mathematical statement is illustrated in Figure 1.1.

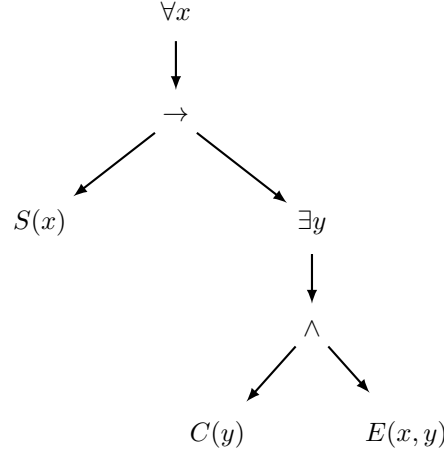


Figure 1.1: The hierarchical parse tree representing the syntactic structure of the formula $\forall x(S(x) \rightarrow \exists y(C(y) \wedge E(x, y)))$.

This hierarchical view is mathematically crucial because it dictates the precise order of operations during semantic evaluation and structural transformations.

1.2 Semantics

While syntax dictates the grammatical structure of formulae, semantics provides their mathematical meaning by evaluating them against a specific interpretation [5].

Definition 1.4 (Structure). *A σ -structure (or model) $\mathcal{M}$ consists of:*

- *a non-empty set $\Delta^{\mathcal{M}}$, called the domain (or universe);*
- *for each predicate symbol P of arity n , a relation $P^{\mathcal{M}} \subseteq (\Delta^{\mathcal{M}})^n$;*
- *for each function symbol f of arity n , a function $f^{\mathcal{M}} : (\Delta^{\mathcal{M}})^n \rightarrow \Delta^{\mathcal{M}}$.*

Definition 1.5 (Interpretation and Satisfaction). *To evaluate formulae containing free variables, we introduce a variable assignment $\beta : \mathcal{V} \rightarrow \Delta^{\mathcal{M}}$, which maps variables to specific domain elements. The satisfaction relation, denoted as $\mathcal{M}, \beta \models \varphi$, is defined inductively:*

- $\mathcal{M}, \beta \models P(t_1, \dots, t_n)$ iff $(t_1^{\mathcal{M}}[\beta], \dots, t_n^{\mathcal{M}}[\beta]) \in P^{\mathcal{M}}$;
- $\mathcal{M}, \beta \models t_1 = t_2$ iff $t_1^{\mathcal{M}}[\beta] = t_2^{\mathcal{M}}[\beta]$;
- $\mathcal{M}, \beta \models \neg \varphi$ iff $\mathcal{M}, \beta \not\models \varphi$;

- $\mathcal{M}, \beta \models \varphi \wedge \psi$ iff $\mathcal{M}, \beta \models \varphi$ and $\mathcal{M}, \beta \models \psi$;
- $\mathcal{M}, \beta \models \exists x \varphi$ iff there exists at least one element $d \in \Delta^{\mathcal{M}}$ such that $\mathcal{M}, \beta[x \mapsto d] \models \varphi$;
- $\mathcal{M}, \beta \models \forall x \varphi$ iff for all elements $d \in \Delta^{\mathcal{M}}$, it holds that $\mathcal{M}, \beta[x \mapsto d] \models \varphi$.

For a sentence φ (which has no free variables, making the assignment β irrelevant), we simply write $\mathcal{M} \models \varphi$ and state that $\mathcal{M}$ is a model of φ .

1.2.1 Concrete Evaluation over a Finite Domain

To illustrate the semantic evaluation process practically, consider a purely relational signature with a unary predicate P and a binary relation R . We can define a finite structure $\mathcal{M}$ with a domain $\Delta^{\mathcal{M}} = \{1, 2, 3\}$. Let the interpretations be:

- $P^{\mathcal{M}} = \{1, 2\}$
- $R^{\mathcal{M}} = \{(1, 2), (2, 3)\}$

Consider the sentence $\exists x(P(x) \wedge \exists y R(x, y))$. To determine if $\mathcal{M}$ is a model for this sentence, we evaluate the truth condition: is there an element in $\{1, 2, 3\}$ that satisfies both P and has an outgoing R -relation? If we assign $x \mapsto 2$, we see that $2 \in P^{\mathcal{M}}$. Furthermore, assigning $y \mapsto 3$ satisfies the relation $(2, 3) \in R^{\mathcal{M}}$. Because we have found a valid assignment, the sentence is true in this structure ($\mathcal{M} \models \varphi$).

Definition 1.6 (Satisfiability and Validity). *A first-order formula φ is classified as:*

- satisfiable if there exists at least one structure $\mathcal{M}$ and assignment β such that $\mathcal{M}, \beta \models \varphi$;
- valid (or a tautology) if $\mathcal{M}, \beta \models \varphi$ holds for every possible structure $\mathcal{M}$ and every assignment β ;
- unsatisfiable if it has no model.

It immediately follows that a formula φ is valid if and only if its negation $\neg\varphi$ is unsatisfiable.

1.3 Decidability and Undecidability

The expressiveness of First-Order Logic makes it an extremely powerful tool for formalizing mathematics and computer science. However, this expressive power comes at a severe computational cost.

1.3.1 Semi-Decidability

Before discussing the absolute limits of FOL, it is crucial to recognize Gödel’s Completeness Theorem (1930) [6], which establishes that FOL is *semi-decidable* (or recursively enumerable). This theorem guarantees that if a formula φ is truly valid, there exists a finite formal proof for it. Consequently, a systematic algorithm can eventually find this proof and confirm its validity in a finite amount of time.

However, the semi-decidability property provides no guarantees for formulae that are *not* valid. If an automated reasoning system attempts to prove an invalid formula, it may search for a proof indefinitely, cycling through infinite domains without ever halting to report a negative result.

Theorem 1.1 (Undecidability of FOL [3, 14]). *The satisfiability problem for full first-order logic is undecidable: there exists no general algorithm that, given an arbitrary FOL sentence, can reliably determine in a finite amount of time whether the sentence is satisfiable.*

This profound result, established independently by Alonzo Church and Alan Turing in the 1930s, places a fundamental theoretical limit on automated reasoning over unrestricted FOL.

This inherent undecidability has directly motivated the systematic study of syntactically restricted *fragments* of FOL. By limiting certain syntactic features—such as the number of available variables, the arity of predicates, or the allowed quantifier prefixes—researchers have identified subclasses of logic whose satisfiability problem *is* fully decidable. The algorithmic generation and semantic testing of these specific decidable fragments lie at the very heart of the research and software developed for this thesis.

Chapter 2

Decidable Fragments

The fundamental undecidability of First-Order Logic necessitates the study of syntactically restricted *fragments* that achieve decidability while retaining enough expressivity for practical applications. This chapter introduces the decidable fragments targeted by our thesis.

Note on function symbols. The decidable fragments studied in this thesis are defined over a *purely relational* signature, i.e., a signature containing only predicate symbols, variables, and possibly constants. Function symbols of arity ≥ 1 are excluded, as they lead to undecidability. All logical analyses described in this thesis therefore operate within purely relational signatures.

2.1 The Two-Variable Fragment

The two-variable fragment of First-Order Logic, denoted by FO^2 , is defined as the set of all FOL formulae that use at most two distinct variable names (typically x and y). Introduced in the context of the decision problem, FO^2 is one of the most prominent decidable fragments, balancing expressive power with a complexity class that remains within NExpTime [8].

The syntax of FO^2 is defined inductively. Let τ be a relational signature and $V = \{x, y\}$ be the set of available variables. The set of FO^2 formulae is defined as the smallest set such that:

Definition 2.1 (Two-Variable Formula).

1. **Atomic:** Every atomic formula $P(t_1, \dots, t_n)$ using variables strictly from V is an FO^2 formula. Additionally, if $v_1, v_2 \in V$, then the equality $v_1 = v_2$ is an FO^2 formula.
2. **Boolean:** If ϕ and ψ are FO^2 formulae, then $\neg\phi$, $\phi \wedge \psi$, and $\phi \vee \psi$ are FO^2 formulae.

3. Quantification:

If ϕ is an FO^2 formula and $z \in \{x, y\}$, then $\exists z\phi$ and $\forall z\phi$ are FO^2 formulae.

2.1.1 Tree Representation and Variable Recycling

From a generative perspective, FO^2 formulae, like those in other fragments, are constructed via a top-down traversal of an Abstract Syntax Tree (AST). However, the generative constraint here differs significantly from the Fluted or Guarded fragments. Instead of maintaining a growing stack of variables (as in FF) or ensuring local variable coverage via guard atoms (as in GF), the AST traversal for FO^2 is globally bounded by the fixed variable pool $V = \{x, y\}$.

This imposes a *variable recycling* mechanism during the tree traversal. When constructing a deep AST branch, quantifiers must continuously reuse x and y , effectively “shadowing” or overwriting the variable bindings from higher up in the tree. For example, a valid FO^2 nested structure might look like the formula in Equation (2.1):

$$\forall x \left(P(x) \vee \exists y (R(x, y) \wedge \exists x (S(y, x))) \right) \quad (2.1)$$

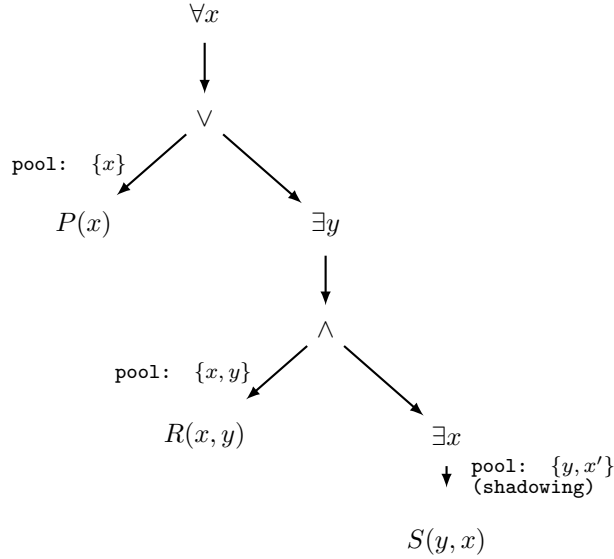


Figure 2.1: Syntactic tree of the FO^2 formula (2.1). The active variable pool is passed down the tree and strictly bounded to $\{x, y\}$. Notice how the innermost existential quantifier over x rebinds the variable (denoted as x'), shadowing the outermost universal binding.

As illustrated in Figure 2.1, the innermost existential quantifier over x rebinds the variable, severing the connection to the outermost universal quantifier. For the generator, this means that tracking the available variables is trivial (it is always a choice between x and y), but ensuring that the generated AST produces meaningful, non-trivial satisfiability challenges requires careful handling of this rebinding mechanism.

2.1.2 Finite Model Property and SAT Generation

A defining characteristic of FO^2 is that it possesses the *finite model property*. Originally established by Mortimer [?], the property states that if an FO^2 formula is satisfiable, it is satisfiable in a finite model. Later, Grädel, Kolaitis, and Vardi [8] sharpened this result, proving that any satisfiable FO^2 formula has a model whose size is bounded exponentially by the size of the formula (specifically, a domain of size at most $2^{O(|\phi|)}$).

This theoretical foundation provides a powerful mechanism for procedural SAT generation. Because we are guaranteed that a finite model suffices, we can generate satisfiable FO^2 instances by constructively evaluating candidate abstract syntax trees against a pre-defined finite model. By fixing a finite domain (a bounded *domain size*) and randomly generating interpretations for the relational vocabulary, a generator can traverse the AST top-down, propagating a set of target variable assignments (i.e., tuples $\langle x, y \rangle$).

At each generative step, an evaluator evaluates the partially constructed formula under the current assignments over the finite model. Subtrees are generated such that they evaluate to true for the specified target configurations, enforcing satisfiability by construction without requiring an external SMT solver to find a model *a posteriori*.

2.1.3 Complexity

The satisfiability problem for FO^2 is known to be **NExpTime-complete** [8]. The high complexity class reflects the expressive power of the fragment, which, despite the two-variable restriction, can still concisely encode exponentially large grids and complex relational structures through variable recycling.

From the perspective of procedural generation, this complexity translates into combinatorial challenges during the target-driven SAT construction. Because universal quantifiers ($\forall$) and negation ($\neg$) require evaluating and complementing variable assignments across the entire finite domain, the number of target assignments can grow rapidly. For instance, complementing a target set requires computing the difference between the full domain cross-product and the current targets.

To mitigate the computational overhead associated with the NExpTime nature of the fragment, practical implementations must employ bound limits (e.g., capping the maximum number of targets propagated) and utilize sparse data structures. This ensures that the SAT construction remains computationally

feasible while still producing logically deep and mathematically valid FO^2 formulae.

2.2 The Fluted Fragment

The landscape of decidable fragments is characterized by the trade-off between expressivity and complexity. By contrast to fragments defined by variable counting (like FO^2), the Fluted Fragment (FF) imposes a restriction on the ordering of variables. This variable-ordering discipline is significant because it preserves predicate arity and formula structure, focusing entirely on the logical application of predicates to variable sequences.

FF acts as a generalization of modal logic, sharing characteristics with the guarded fragments; however, it allows the negation of relation atoms, enabling the capture of enriched modal logics. Originally stemming from the work of Quine [12], the fragment’s decidability is modernized and formalized by Pratt-Hartmann, Szwaig, and Tendera [11].

The fundamental constraint of FF is the *suffix property*. An atomic formula is fluted if its arguments form a contiguous suffix of the variables currently in scope.

Definition 2.2 (Fluted Formula).

The set of fluted formulae over X_i (where $0 \leq i \leq m$) is defined inductively:

1. **Atomic:**

For any n -ary predicate $P \in \mathcal{P}$ such that $n \leq i$, the atomic formula $P(x_{i-n+1}, \dots, x_i)$ is fluted over X_i .

2. **Boolean:**

If ϕ and ψ are fluted over X_i , then $\neg\phi$, $\phi \wedge \psi$, $\phi \vee \psi$, and $\phi \Rightarrow \psi$ are fluted over X_i .

3. **Quantification:**

If ϕ is fluted over X_{i+1} , then $\exists x_{i+1}\phi$ and $\forall x_{i+1}\phi$ are fluted over X_i .

This inductive structure entails that the quantification step expands the active variable scope, while atomic leaves are strictly bounded to the most recently introduced variables.

2.2.1 Tree Representation and the Scope Stack

The structure of a fluted formula is best understood through its Abstract Syntax Tree (AST). The variable-ordering discipline of FF naturally translates to a stack-based traversal during top-down generation.

Unlike FO^2 , which recycles a fixed pool of variables, building an AST for the Fluted Fragment requires maintaining a dynamic “scope stack” of quantified variables that is passed down to child nodes. Each quantifier node conceptually pushes a new variable onto this stack.

Consider the following valid fluted formula:

$$\forall x_1 \forall x_2 \left(P(x_1, x_2) \wedge \exists x_3 (R(x_2, x_3)) \right) \quad (2.2)$$

Its syntactic structure can be visually represented as an AST, as shown in Figure 2.2.

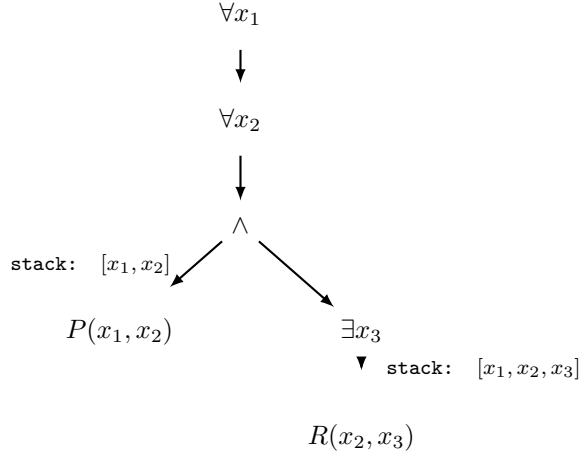


Figure 2.2: Syntactic tree of the fluted formula (2.2). The active variable stack is passed down the tree, and atomic leaves deterministically consume a contiguous suffix of it.

When the AST traversal reaches a leaf node (an atomic formula), the suffix property acts as a strict generative constraint. For instance, when generating the atom R of arity 2 in Figure 2.2, the active stack is $[x_1, x_2, x_3]$. To comply with FF rules, the atom must exclusively consume the contiguous suffix (x_2, x_3) . Attempting to construct an atom such as $Q(x_1, x_3)$ at that exact position would invalidate the formula. This inherent top-down predictability allows for an efficient, stack-driven generation strategy.

2.2.2 Complexity

The general Fluted Fragment exhibits non-elementary complexity, which scales heavily with the number of variables m [11]. This theoretical landscape poses a severe computational challenge, as attempting to prove unsatisfiability in FF can require exploring an enormously vast state space. In the context of this thesis, this justifies the necessity of our generation strategy: by algorithmically constructing structured FF formulae, we provide theorem provers with strictly compliant input to benchmark their limits against this non-elementary complexity.

2.3 The Guarded Fragment

The Guarded Fragment (GF), introduced by Andr  ka, van Benthem, and N  meti [1], imposes structural constraints on the interaction between quantifiers and their sub-formulae. Quantification must be "guarded" by atomic formulae, which effectively restricts the domain of the quantifier to tuples satisfying a specific relational constraint.

The syntax of GF is defined inductively. Let τ be a relational signature. The set of guarded formulae is defined as the smallest set such that:

Definition 2.3 (Guarded Formula).

1. **Atomic:**

Every atomic formula $P(t_1, \dots, t_n)$ is a guarded formula.

2. **Boolean:**

If ϕ and ψ are guarded formulae, then $\neg\phi$ and $\phi \wedge \psi$ are guarded formulae.

3. **Quantification:**

Let $\phi(\bar{x}, \bar{y})$ be guarded and $G(\bar{x}, \bar{y})$ be an atomic guard containing all free variables of ϕ . Then $\exists \bar{y}(G(\bar{x}, \bar{y}) \wedge \phi)$ and $\forall \bar{y}(G(\bar{x}, \bar{y}) \Rightarrow \phi)$ are guarded formulae.

GF satisfies the *tree model property* [1], meaning every satisfiable guarded formula can be satisfied in a tree-structured model. This is the cornerstone of its decidability [7].

2.3.1 Tree Representation and Guard Generation

From a generative perspective, constructing a guarded formula requires a significantly different Abstract Syntax Tree (AST) traversal compared to the Fluted Fragment. While FF maintains a linear stack to enforce the suffix property, the generation of GF formulae relies on tracking a dynamic set of currently active free variables. In our top-down AST generation, whenever a quantifier over a set of new variables $\bar{y}$ is introduced, the fragment's rules dictate that a guard atom $G(\bar{x}, \bar{y})$ must be explicitly constructed, where $\bar{x}$ represents the variables currently in the active scope. To satisfy this, the generator must query the available vocabulary for a predicate whose arity exactly matches the combined size of the active scope and the newly bound variables.

Consequently, a quantified node in the AST is not a simple unary wrapper but a composite structure. For an existential quantification, the AST node branches into an $\wedge$ connective (or an $\rightarrow$ connective for universal quantification), with the specifically tailored guard atom G as its left child and the recursive sub-formula ϕ as its right child. The traversal then proceeds to generate the body of ϕ with the expanded active scope $\bar{x} \cup \bar{y}$. If no predicate of the required arity is available in the vocabulary to form a valid guard, the branch is deemed invalid, and the generator must backtrack by triggering a retry exception.

An elegant example of this generative structure is shown in Equation (2.3):

$$\exists x_2 \left(G(x_1, x_2) \wedge \forall x_3 (H(x_1, x_2, x_3) \rightarrow (P(x_2) \vee S(x_1, x_3))) \right) \quad (2.3)$$

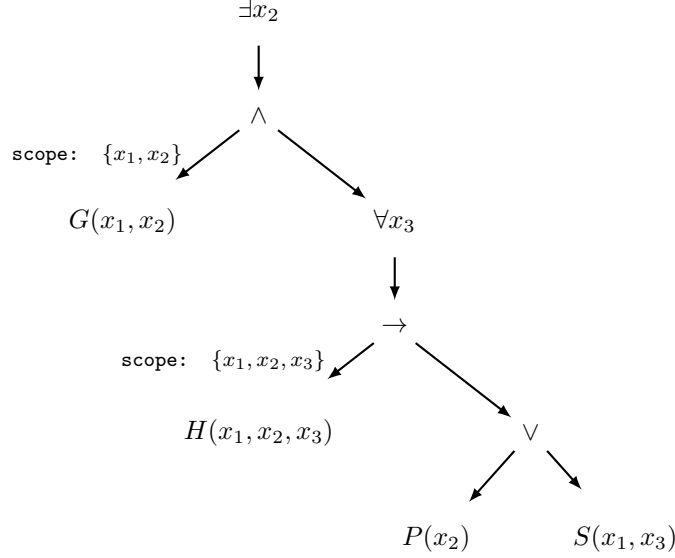


Figure 2.3: Syntactic tree of the guarded formula (2.3) assuming an initial active variable x_1 . The dynamic active scope grows as quantifiers are introduced. Each quantification spawns a composite node structure where an explicit guard atom shields the sub-formula, strictly consuming the current variable scope.

As illustrated in Figure 2.3, this explicit structural constraint ensures that every bound variable is immediately shielded by a guard atom (G and H , respectively) that spans the entire active scope. The magnificence of this generative approach lies in its structural determinism: the complex syntactic restrictions of the Guarded Fragment are not enforced *post-hoc* through rejection sampling, but are structurally guaranteed *by design* during the AST traversal. The tree is forced to interleave logical connectives with specific atoms from the vocabulary whose arities perfectly match the dynamically expanding scope, thus translating theoretical logical boundaries into an elegant and highly constrained algorithmic generation pipeline.

2.3.2 Complexity

The satisfiability problem for the Guarded Fragment is ExpTime-complete [7]. This result highlights a significant increase in expressive power compared to

basic propositional modal logic (which is typically PSpace-complete), while remaining much more tractable than the non-elementary complexity of the Fluted Fragment. Benchmarking provers against GF instances is particularly valuable for observing how solvers handle deeply nested local constraints without being overwhelmed by global variable dependencies.

2.4 The Unary Negation Fragment

The Unary Negation Fragment (UNFO), introduced by ten Cate and Segoufin [13], restricts the scope of negation rather than the quantifier pattern. While the Guarded Fragment restricts the placement of quantifiers to achieve a tree-like model, UNFO allows arbitrary quantification but isolates the primary source of undecidability in FOL: the unrestricted use of negation over multiple variables.

The syntax of UNFO is defined inductively. Let τ be a relational signature. The set of UNFO formulae is defined as the smallest set such that:

Definition 2.4 (Unary Negation Formula).

1. **Atomic:**

Every atomic formula is an UNFO formula.

2. **Boolean:**

If ϕ and ψ are UNFO formulae, then $\phi \wedge \psi$ and $\phi \vee \psi$ are UNFO formulae.

3. **Negation:**

If ϕ is an UNFO formula such that $|\text{free}(\phi)| \leq 1$, then $\neg\phi$ is an UNFO formula.

4. **Quantification:**

If ϕ is an UNFO formula, then $\exists x\phi$ and $\forall x\phi$ are UNFO formulae.

Decidability in UNFO is maintained by confining the complexity of negation to formulae with at most one free variable, effectively preventing the construction of undecidable contradictions in higher-arity predicates [13].

2.4.1 Tree Representation and Variable Dependencies

From a structural perspective, unlike the Fluted Fragment (which relies on a strict variable stack) or the Guarded Fragment (which forces guard atoms at every quantification step), UNFO is highly permissive with positive connectives and quantifiers. The structural "bottleneck" occurs exclusively at the negation nodes.

Consider the following valid UNFO formula:

$$\forall x \left(P(x) \vee \exists y (R(x, y) \wedge \neg Q(y)) \right) \quad (2.4)$$

In this formula, the negation is applied solely to the sub-formula $Q(y)$, which contains exactly one free variable (y). This satisfies the unary restriction. Its AST representation is shown in Figure 2.4.

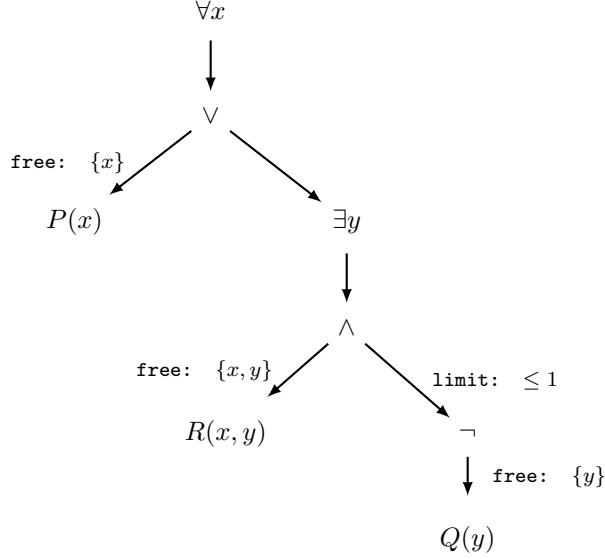


Figure 2.4: Syntactic tree of the valid UNFO formula (2.4). The negation node acts as a strict structural filter, ensuring that the sub-tree below it depends on at most one free variable.

As illustrated in Figure 2.4, the positive branches (like the one leading to $R(x, y)$) can freely accumulate multiple variables in their scope. However, when the AST branches into a negation node, the set of free variables must be aggressively restricted.

Conversely, consider the following invalid formula:

$$\forall x \exists y (\neg R(x, y)) \quad (2.5)$$

This construction strictly violates the UNFO rules because the sub-formula under the negation, $R(x, y)$, contains two free variables (x and y). This creates a structural topology where branches underneath a negation node are inherently isolated in terms of variable dependencies. This architectural behavior perfectly mirrors the theoretical foundation of UNFO: it allows complex positive relations but forbids non-local negated structures, neutralizing the main trigger of undecidability in unrestricted FOL.

2.4.2 Complexity

The complexity of the satisfiability problem for UNFO is 2ExpTime-complete in the general case, but it drops to ExpTime-complete if the maximum arity of the predicates in the signature is fixed [13]. This makes UNFO an incredibly powerful theoretical tool: it offers a much larger expressive footprint than GF—allowing properties over arbitrarily many variables without guard atoms—while sitting within a comparable, well-understood complexity class.

2.5 Propositional Modal Logic and the Standard Translation

While the fragments discussed thus far (FO^2 , FF, GF, UNFO) are defined purely within the vocabulary of First-Order Logic, their origins, properties, and constraints are deeply intertwined with Propositional Modal Logic (ML). Historically, modal logic was developed to reason about necessity and possibility, but from a modern computational perspective, it is best understood as a robustly decidable, tree-like logic for reasoning about relational structures, often evaluated over Kripke semantics.

The fundamental insight that connects ML to the decidable fragments of FOL is that modal operators can be viewed as “macros” for specific, heavily restricted patterns of first-order quantification. This connection establishes modal logic as the baseline or “minimal” decidable fragment, out of which fragments like GF and UNFO evolved as expressive generalizations.

2.5.1 The Standard Translation

The formal bridge between Propositional Modal Logic and First-Order Logic is the *Standard Translation* (ST). This translation maps any modal formula into an equivalent FOL formula over a relational signature containing only unary predicates (corresponding to modal propositions) and a single binary predicate R (corresponding to the accessibility relation between worlds).

Let $\mathcal{P}$ be a set of propositional variables. The standard translation $ST_x(\phi)$ of a modal formula ϕ evaluated at a world x is defined inductively:

Definition 2.5 (Standard Translation).

1. **Atomic:**
For any propositional variable $p \in \mathcal{P}$, $ST_x(p) = P(x)$, where P is a unary predicate.
2. **Boolean:**
 $ST_x(\neg\phi) = \neg ST_x(\phi)$ and $ST_x(\phi \wedge \psi) = ST_x(\phi) \wedge ST_x(\psi)$.
3. **Existential Modality (Diamond):**
 $ST_x(\Diamond\phi) = \exists y (R(x, y) \wedge ST_y(\phi))$.

4. *Universal Modality (Box):*

$$ST_x(\Box\phi) = \forall y(R(x, y) \Rightarrow ST_y(\phi)).$$

As evident from the **Existential** and **Universal** rules, the translation of modal operators inherently produces guarded quantifiers. The binary relation $R(x, y)$ acts as a strict local guard, evaluating the sub-formula $ST_y(\phi)$ only in worlds y that are directly accessible from the current world x .

2.5.2 Tree Representation and Variable Alternation

A remarkable structural property of the Standard Translation is its extreme parsimony with variables. Although the translation of nested modal operators (e.g., $\Diamond\Box\Diamond p$) conceptually moves through a sequence of distinct worlds, the resulting FOL formula does not require an unbounded number of variable names.

From an Abstract Syntax Tree (AST) perspective, building the translation requires tracking the current context node (the active world). When a modal operator expands the AST by introducing a quantifier over a new world, the target variable only needs to be distinct from the immediately preceding active world. Once the traversal moves deeper into the tree, the variables from higher up in the hierarchy are no longer actively referenced in the local relational scope and can be safely overwritten.

Consequently, any modal formula can be translated using exactly two variables—conventionally denoted as w_1 and w_2 —which strictly alternate as the depth of the modal nesting increases. For instance, consider the standard translation of the modal formula $\Box(p \vee \Diamond q)$ evaluated at an initial world w_1 , as shown in Equation (2.6):

$$\forall w_2 \left(R(w_1, w_2) \rightarrow (P(w_2) \vee \exists w_1 (R(w_2, w_1) \wedge Q(w_1))) \right) \quad (2.6)$$

As illustrated in Figure 2.5, this variable alternation relies on local rebinding rather than an expanding variable stack. This structural behavior demonstrates why standard Modal Logic can be naturally embedded into FO^2 : it relies exclusively on a two-variable pool with recycling. Furthermore, because its quantifiers are always structurally shielded by the accessibility relation R , it perfectly embeds into the Guarded Fragment (GF). Finally, because its propositional properties are strictly unary ($P(x)$ and $Q(x)$) and are evaluated locally at the context nodes, it inherently satisfies the restrictions of the Unary Negation Fragment (UNFO).

2.5.3 Complexity

The satisfiability problem for basic propositional modal logic is PSpace-complete [?]. While translating modal formulae into First-Order Logic via the standard translation produces formulae within the Guarded Fragment (and the Two-Variable Fragment), the native complexity of modal logic is lower than the ExpTime-completeness of GF. This makes modal logic an excellent baseline for

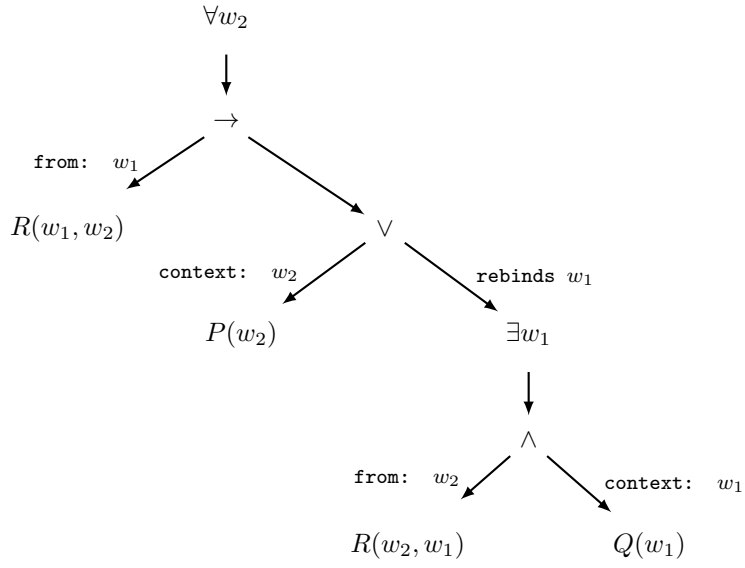


Figure 2.5: Syntactic tree of the translated modal formula (2.6). The translation strictly alternates between w_1 and w_2 . When the inner diamond operator ($\Diamond q$) is expanded, it reuses w_1 for its target world, completely shadowing the original initial world.

benchmarking theorem provers: it allows for deep, highly nested tree-like models that stress a prover’s memory management, while theoretically remaining within a strictly smaller complexity class than its encompassing first-order fragments.

2.5.4 Summary

In conclusion, Propositional Modal Logic serves as the central node connecting all the fragments studied in this chapter.

- The **Two-Variable Fragment** generalizes ML by abandoning the guard relation entirely, relying instead on the strict two-variable limit and recycling.
- The **Fluted Fragment** generalizes ML by viewing the sequence of modal transitions as a strict variable-ordering discipline (the scope stack). Crucially, unlike GF, FF allows the negation of relational atoms, enabling the embedding of enriched modal logics such as Boolean modal logic.
- The **Guarded Fragment** generalizes ML by maintaining the strict guard mechanism but extending it to relations of arbitrary arity.
- The **Unary Negation Fragment** generalizes ML by lifting restrictions on positive quantification entirely, demanding only that negation remains structurally simple (unary).

By understanding the standard translation, we obtain a clear topological map of the boundaries of decidability in First-Order Logic. Modal logic acts as the minimal intersection point, while FO^2 , FF, GF, and UNFO represent different structural paths to extend its expressivity without crossing the threshold into undecidability. This provides the necessary theoretical foundation for the generation algorithms proposed in this thesis.

Chapter 3

System Architecture

Developing an automated generator for complex logical fragments requires efficiency, scalability, and highly maintainable software architecture. This chapter details the engineering principles, structural design, and implementation strategies underlying the developed system.

To adhere to professional software engineering standards and ensure long-term extensibility throughout the software development lifecycle, the system's architecture leverages established object-oriented design patterns such as the Factory, Template Method, Context Object, and Visitor patterns. By strictly enforcing the separation of concerns, safe memory management, and modularity, the design guarantees that the generated formulas are not only syntactically well-formed but also mathematically **valid by construction**.

The following sections systematically explore the core layers of the application. First, we discuss the lexical abstraction and the foundational data structures based on the Abstract Syntax Tree (AST). Next, we analyze the core generation engine and its algorithmic orchestration.

3.1 Lexical Abstraction

The foundational layer of the architecture handles the definition of the system's logical alphabet. Every syntactic element is encapsulated within the `Symbol` structure and strictly typed through the `SymbolType` enumeration. This system systematically defines all standard logical constructs:

punctuation $((,))$, negation $(\neg)$, connectives $(\wedge, \vee, \rightarrow, \leftrightarrow)$, quantifiers $(\forall, \exists)$, equality $(=)$, variables $(x_1, x_2, ..)$, and predicates (A_1 for unary predicates, B_1 for binary predicates, ..).

To instantiate these entities, the creation of symbols is delegated to dedicated static **factory methods**, such as `Symbol::var()` for variables, or `Symbol::pred()` for predicates. The implementation of this creational pattern guarantees en-

capsulation and the enforcement of domain invariants right at the moment of creation.

The internal representation is built upon an Abstract Syntax Tree (AST). This hierarchical data structure intrinsically ensures that generated formulas are well-formed *by construction*.

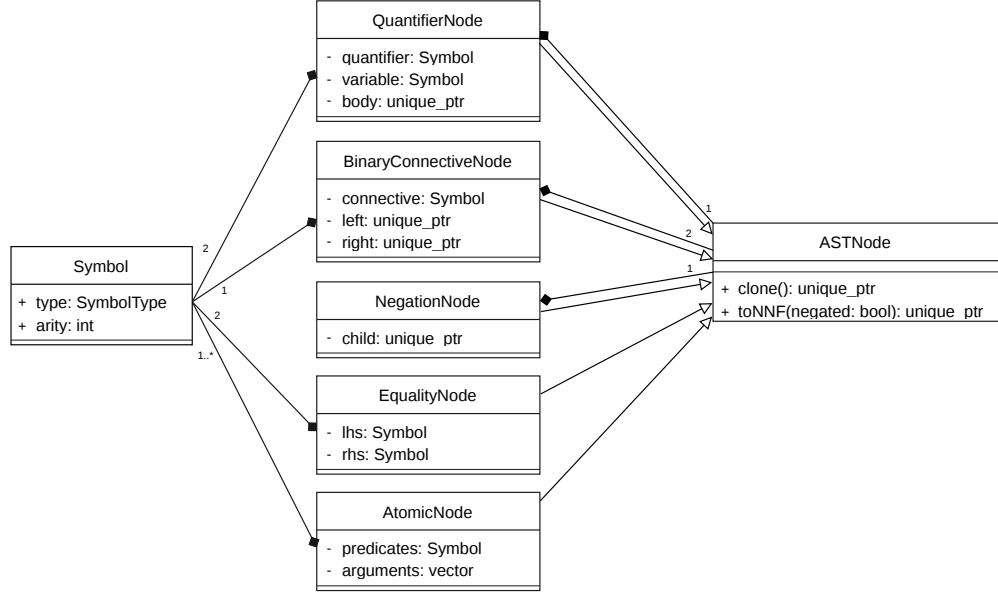


Figure 3.1: Class Diagram representing the AST architecture. The abstract base class `ASTNode` acts as the root, with specific final classes implementing the concrete nodes. Composition relationships highlight how nodes manage their children through exclusive ownership.

The tree’s structural integrity relies on strict ownership and safe memory management. Complex nodes (such as `NegNode`, `BinaryConnNode`, and `QuantifierNode`) encapsulate their children via exclusive smart pointers (`std::unique_ptr`). This design perfectly mirrors the compositional nature of logical formulas: parents exclusively own their branches, guaranteeing automatic memory cleanup upon root destruction without the runtime overhead of a garbage collector.

This memory model synergizes with pure **polymorphism** to execute complex logical transformations, such as the conversion to **Negation Normal Form** (`toNNF()`). Relying on polymorphic recursion, each node independently applies type-specific rules to dynamically generate a structurally modified branch. For instance, an implication within a `BinaryConnNode` dynamically rewrites itself into an equivalent disjunction (OR) with a negated left child, seamlessly

propagating the transformation down its exclusively owned subtrees.

3.2 Formula Generation

Once the logical alphabet and the structural foundation (AST) are defined, the system requires a robust engine to dynamically assemble these components into well-formed logical expressions. This responsibility is delegated to the **FormulaBuilder**, a core component designed specifically to orchestrate the generation of the Abstract Syntax Tree.

From an architectural standpoint, this component is driven by the **Template Method** design pattern to balance shared orchestration logic with strict fragment-specific syntactic rules. The abstract base class **FormulaBuilder** establishes the invariant algorithmic skeleton: it handles the recursive descent mechanism, the retry loops, and the pre-processing configurations. However, it intentionally defers the instantiation of leaf nodes and the logic for satisfiability to its derived classes through pure virtual methods, specifically `buildAtomic()`, `generateSAT()`, and `buildComponentUNSAT()`.

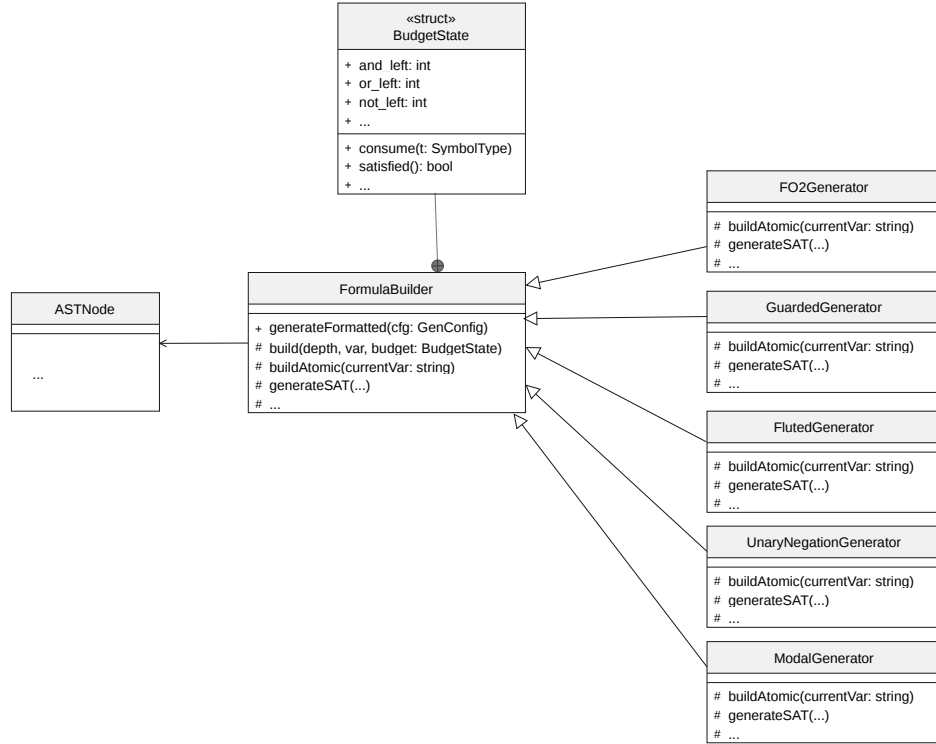


Figure 3.2: UML Class Diagram illustrating the Template Method pattern. **FormulaBuilder** defines the algorithmic skeleton, while concrete generator classes implement the abstract hooks to enforce their specific syntactic rules without duplicating the orchestration logic.

As illustrated in the class diagram (Figure 3.2), concrete implementations—such as the **FO2Generator**, **GuardedGenerator**, and **ModalGenerator**—act as specific strategies, injecting their domain-specific rules into these structural hooks. This polymorphic approach guarantees that each logical fragment can rigorously enforce its own restrictions without altering the central tree-building engine.

To manage the complexity of the recursive generation without heavily polluting method signatures, the architecture implements the **Context Object** pattern through a nested **BudgetState** structure. This state object encapsulates a mutable context for a single building session, dynamically tracking the remaining allowance for connectives and quantifiers. The state semantics are strictly defined: a value of -1 indicates an unconstrained resource, 0 completely forbids the usage of a specific symbol, and any value greater than 0 represents a limited resource that is meticulously decremented down the recursive call stack.

Because logic generation with exact structural constraints can be mathematically unpredictable, the system is engineered for high resilience. The main entry point, `generateFormatted()`, wraps the entire generation process in a retry loop protected by a try-catch mechanism. If a partially generated AST violates the strict limits defined in the `BudgetState`, the local scope throws a custom `BudgetRetryException`. The orchestrator immediately catches this exception, safely discards the invalid tree, and restarts the generation process up to a predefined threshold. This design guarantees that the final output either perfectly satisfies the configuration object constraints or gracefully terminates with an `std::invalid_argument` exception if the requested shape is fundamentally unsatisfiable.

3.3 The FO2 SAT Model and Visitor

To generate logical formulas that are strictly satisfiable (SAT) within the two-variable fragment (FO2), the system’s architecture must transition from purely syntactic assembly to active semantic evaluation. This complex responsibility is orchestrated by the `FO2SATGenerator`, which extends the base `FO2Generator` to inject mathematical validation directly into the generation pipeline. From a software design perspective, this process relies on the strict separation of concerns between the structural representation of the formula, the mathematical domain, and the evaluation algorithm.

The mathematical ground truth required for semantic validation is encapsulated within the `FiniteModel` class. Operating as a Domain Model, this component dynamically generates and stores randomized boolean interpretations for both unary and binary predicates over a specified finite domain size. By abstracting the domain logic, the model provides a clean, stateless interface (`evalAtom()`) to query the truth value of any given assignment without exposing its internal combinatorial matrices.

The most critical architectural challenge in this phase is traversing and evaluating the Abstract Syntax Tree without polluting the pure data structure of the nodes with domain-specific semantic logic. The architecture elegantly resolves this by implementing the **Visitor Pattern**. The `FO2Evaluator` class acts as the concrete visitor, realizing the `ASTVisitor` interface to enable double-dispatch traversal across the tree. As it visits specific node types—such as `AtomicNode`, `QuantifierNode`, or `BinaryConnNode`—the evaluator dynamically computes their boolean truth values against the current variable assignment and the referenced `FiniteModel`.

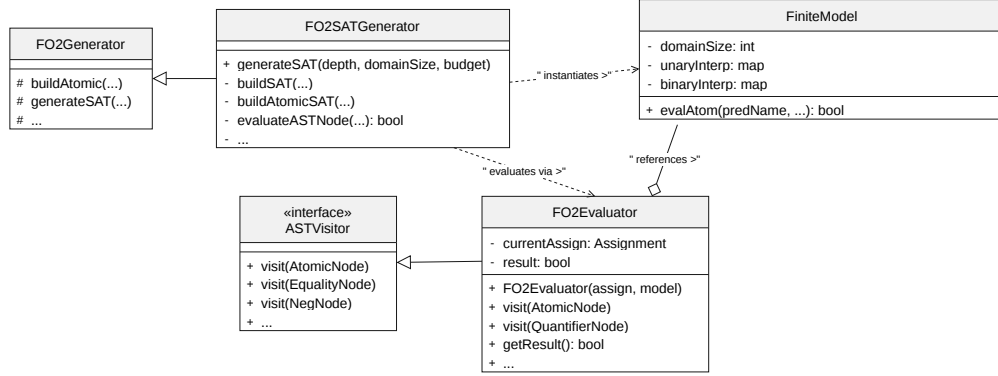


Figure 3.3: UML Class Diagram detailing the semantic SAT generation architecture. The **FO2SATGenerator** orchestrates the **FiniteModel**. It relies on the **Visitor Pattern** via the **FO2Evaluator** to safely traverse the AST and compute semantic truth values without altering the underlying node structures.

As depicted in Figure 3.3, the **FO2SATGenerator** acts as the central coordinator of these patterns. During the recursive construction of a formula, it acts as a client that proposes candidate sub-trees and immediately validates them by instantiating an **FO2Evaluator**. If a generated branch does not satisfy the semantic targets dictated by the **FiniteModel**, the generator leverages this immediate feedback to dynamically adjust the structure, such as wrapping the branch in a **NegNode** or falling back to a tautology, ensuring that the final output is structurally solid and mathematically satisfiable by construction.

3.4 External Verification: The Vampire Actor

While the internal generation engine handles the rigorous construction of syntactically valid formulas, ensuring the satisfiability (SAT) of complex fragments such as the Fluted fragment, often requires a “generate-and-test” approach. Within this system’s architecture, the Automated Theorem Prover (ATP) **Vampire** is modeled as an independent, out-of-process *Actor* that serves as an external semantic oracle.

Rather than tightly coupling the solver’s complex inference logic to the generation engine, the architecture interfaces with this external actor through a dedicated proxy component: the **VampireRunner** class. This component acts as an adapter bridging the internal C++ application with the external operating system environment. Its primary responsibilities include:

- **Data Serialization:** Receiving the internally generated AST and formatting it into the standard TPTP (Thousands of Problems for Theorem Provers) language.

- **Process Orchestration:** Safely managing temporary file I/O and spawning a separate OS-level sub-process to execute the Vampire binary with strict time and memory limits.
- **Result Parsing:** Capturing the standard output via pipes and extracting the standard SZS status (e.g., *Satisfiable*, *Unsatisfiable*, *Timeout*) to feed the result back into the application’s control flow.

By treating Vampire as an external actor, the architecture maintains strict modularity and separation of concerns. The generation pipeline can rapidly produce candidate formulas (e.g., in “free” mode) and immediately delegate the heavy lifting of semantic verification to the solver, which filters out invalid instances.

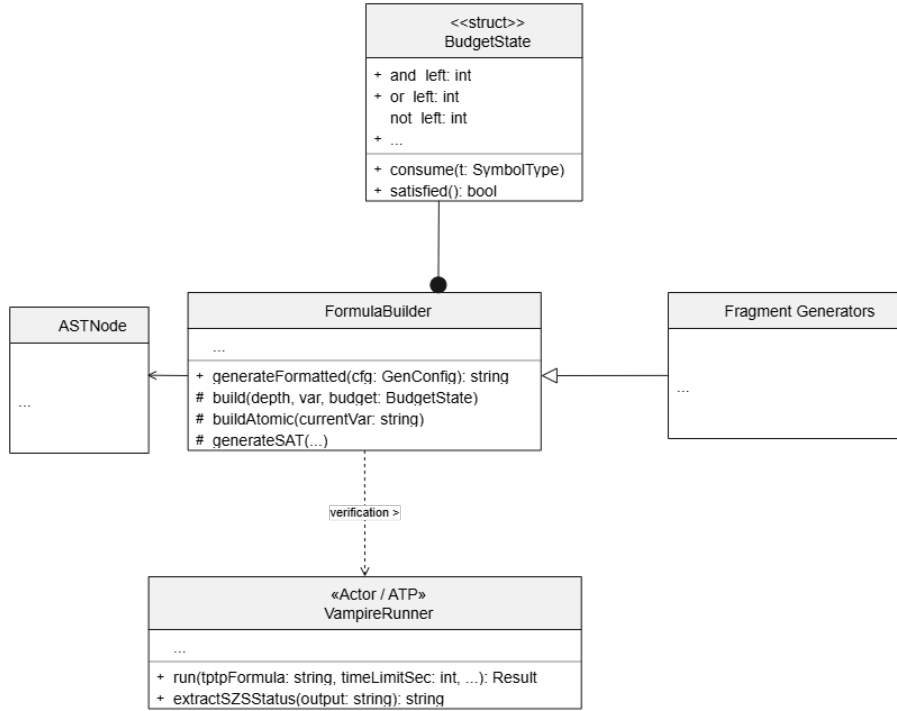


Figure 3.4: UML Component Diagram illustrating the integration of the Vampire Actor. The **VampireRunner** class acts as the structural bridge between the internal generation pipeline and the external ATP process, parsing the SZS output to validate formula satisfiability.

While this section defines Vampire’s structural role and architectural integration as a verification oracle, a comprehensive theoretical analysis of its

underlying logical calculus, resolution techniques, and its central role in the dedicated testing framework will be thoroughly discussed later in Chapter 5.

3.5 Architecture Overview

Having detailed the individual structural and behavioral components of the system, this section synthesizes the global software architecture. The application operates as a unidirectional pipeline characterized by a strict separation of concerns among the configuration layer, the generation engine, and the mathematical evaluation domain.

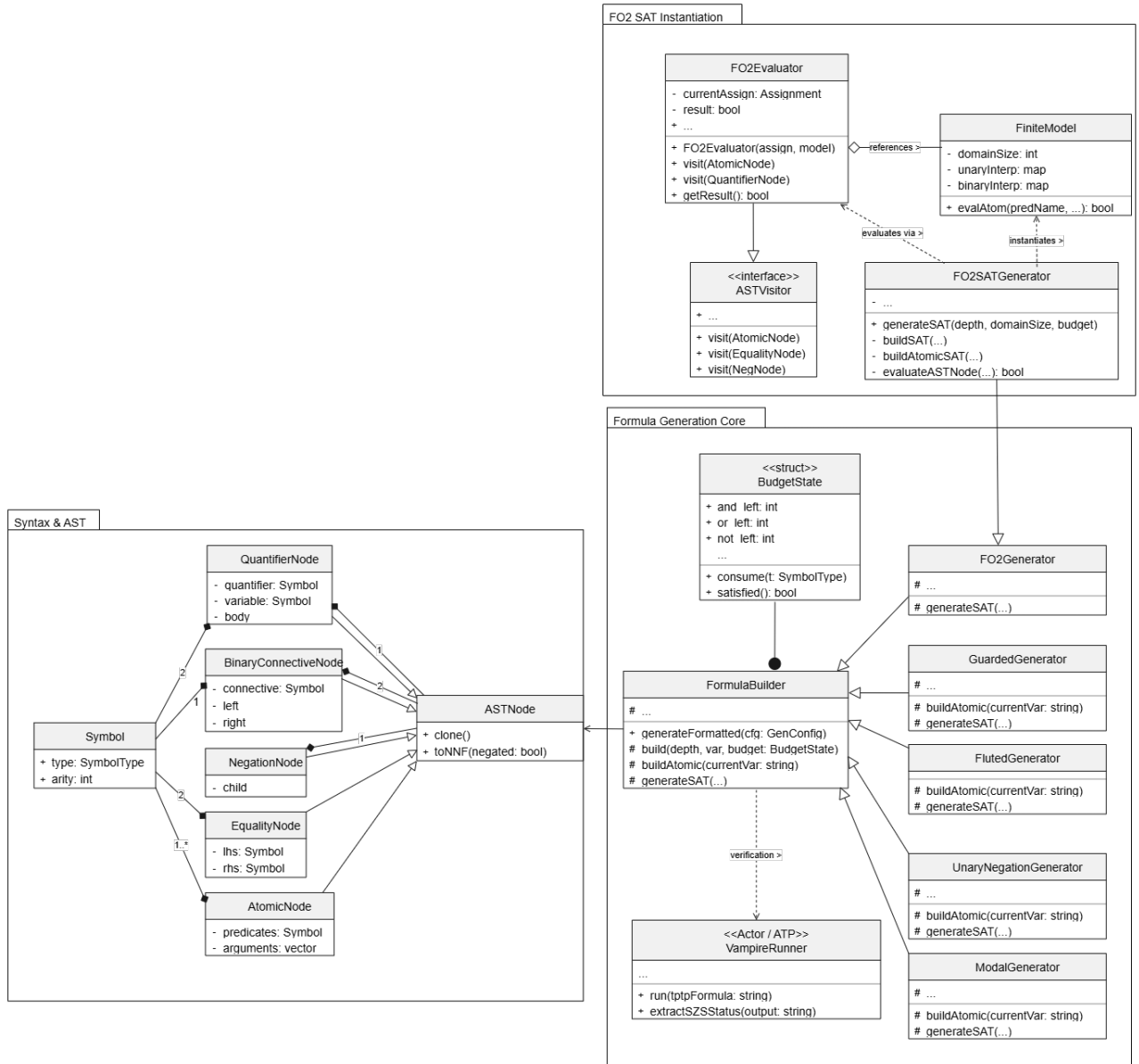


Figure 3.5: Global System Architecture. The diagram illustrates the complete data flow.

3.6 System Interface and CLI

While the core generation engine handles the rigorous construction of logical formulas, the system requires a presentation layer to interact with the end user and external tools. This is achieved through a Command Line Interface (CLI) that acts as the entry point and configuration orchestrator for the entire system.

The CLI is designed to decouple user input from the internal generation logic. When a generation command is issued, the interface parses various execution arguments such as the targeted logical fragment (e.g., FO2, Guarded, Fluted), the maximum abstract syntax tree depth, the random seed for reproducibility, and specific structural constraints. These inputs are not passed raw to the internal logic; instead, they are sanitized and encapsulated into a structured `GenConfig` object.

This configuration object initializes the `BudgetState`, directly influencing the recursive generation process. Furthermore, the CLI strictly formats the generated formulas into standard logical notation via the `generateFormatted()` entry point. By outputting well-formed strings to standard output or safely routing them to dedicated output directories.

3.6.1 Application Controller

Beyond simple parameter parsing, the main entry point serves as an Application Controller that orchestrates the entire verification and testing lifecycle. Depending on the parsed flags, the CLI dynamically routes the execution flow into distinct operational modes:

- **Standard Generation:** Instantiates the specific fragment generator (e.g., `FO2Generator`, `FlutedGenerator`) and executes the generation loop, actively managing file I/O and retrying up to a maximum threshold if the requested budget constraints fail during AST construction.
- **Dataset Orchestration:** Triggers the `generateDatasets` routine to automatically construct massive, categorized corpora of logical formulas.
- **Automated Benchmarking:** Invokes the benchmarking and timeout analysis workflows, seamlessly passing the `VampireRunner` instance to evaluate previously generated datasets.

3.6.2 Self-Documenting Interface

To ensure professional usability and ease of integration into larger automated workflows, the CLI is fully self-documenting. Invoking the application with the `--help` flag provides a comprehensive overview of the available commands, configuration limits, and structural options. A conceptual snippet of this interface is illustrated in Figure 3.6.

```
Command Line Interface

USAGE: ./DecidableFragmentsGen <options>

BASIC OPTIONS:
  --fragment <name>
  --mode <mode>
  --domain-size <n>
  --depth <n>
  --count <n>
  --seed <n>
  --help

FORMULA STRUCTURE OPTIONS:
  --and <n|min:max>
  --or <n|min:max>
  --not <n|min:max>
  --exists <n|min:max>
  --forall <n|min:max>
  --implies <n|min:max>
  --eq <n|min:max>

TESTING & DATASETS OPTIONS:
  --run-benchmarks <path>
  --generate-datasets
  --run-timeout-analysis

VOCABULARY OPTIONS:
  --preds <count>/<arity>
  --arity <k|mixed>

OUTPUT OPTIONS:
  --output <format>

TRANSFORMATION OPTIONS:
  --transform <mode>

VERIFICATION OPTIONS:
  --verify
  --vampire-path <path>
  --vampire-timeout <n>
```

Figure 3.6: The compact Command Line Interface help menu, displaying the configuration options grouped by functional category.

Chapter 4

Implementation

The core contribution of this thesis is the implementation of a modular, parametric system for generating random formulae that belong to specific decidable fragments of First-Order Logic. Generating structured logical formulae frequently results in ill-formed or trivially unsatisfiable expressions. To overcome this, our implementation relies on a highly configurable, probabilistic architecture centered around a robust internal data structure.

4.1 Development Environment

The development of the formula generation system is implemented in **C++**, utilizing the most modern standard available at the time of inception: **C++23**.

The choice of C++23 allowed the codebase to benefit from modern programming paradigms, prioritizing safety without sacrificing zero-cost abstractions. Several key modern C++ features were heavily utilized throughout the codebase:

- **Smart Pointers for Memory Safety:** Raw pointers were strictly avoided. The standard library’s `<memory>` header, specifically `std::unique_ptr`, was employed to manage the lifecycle of polymorphic objects, guaranteeing the absence of memory leaks during the recursive generation of trees.
- **Move Semantics (`std::move`):** To avoid expensive deep copies of tree branches during AST construction, `std::move` was consistently used to transfer ownership of generated sub-formulas between nodes and functions.
- **Modern Attributes:** The `[[nodiscard]]` attribute was extensively applied to pure virtual interfaces and builder methods (e.g., in the `FormulaBuilder` class) to ensure that generated sub-trees or vital state queries were not accidentally ignored by the compiler.
- **Lambda Expressions:** Localized algorithmic logic, such as the final formatting of formulas into TPTP syntax or forced quantification steps,

was encapsulated in stateful lambdas (e.g., `auto finalize = [&](...)`), improving code locality and readability.

- **Exception Handling:** Custom exception types, such as `BudgetRetryException`, were implemented to elegantly backtrack during randomized generation if a specific architectural constraint (or budget) could not be met.

We use **CMake** as its build system, ensuring cross-platform compilation compatibility with modern toolchains (GCC, Clang, or MSVC) capable of supporting the C++23 standard.

4.2 The Abstract Syntax Tree (AST)

The fundamental data structure of the system is the Abstract Syntax Tree (AST), which translates the formal grammar of First-Order Logic into a strictly typed C++ class hierarchy. As requested by the architectural design, the implementation relies on modern C++ features, primarily `std::unique_ptr`, to ensure exclusive memory ownership and automatic bottom-up deallocation without garbage collection overhead.

4.2.1 Node Hierarchy and Interfaces

The tree is built around the pure virtual base class `ASTNode`. This class exposes a unified interface that all concrete logical nodes (`AtomicNode`, `EqualityNode`, `NegNode`, `BinaryConnNode`, and `QuantifierNode`) must implement:

- `clone()`: Performs a recursive deep copy of the tree branch.
- `toString()` and `toTPTP()`: Handle the string serialization of the formula. The latter maps the internal AST representation to the standard TPTP format (e.g., translating existential quantifiers to `?` and universal to `!`, while ensuring variables are correctly capitalized).
- `accept(ASTVisitor&)`: Acts as the entry point for the Visitor design pattern, allowing external components (like SAT model generators) to traverse the AST without modifying the node classes.

4.2.2 The Negation Normal Form (NNF)

To seamlessly interface with external Theorem Provers, the AST includes a built-in transformation to Negation Normal Form (NNF), and the conversion is handled polymorphically.

The procedure recursively eliminates implications and pushes negations down to the atomic leaves using De Morgan's laws, gracefully flipping binary connectives and quantifiers on the fly. The underlying logic is formalized in Algorithm 1.

Algorithm 1: NNF Transformation for AST Nodes

```
1 function toNNF( $n$ : Node,  $neg$ :  $\mathbb{B}$ ): Node
2   if  $n = (L \rightarrow R)$  then
3      $n' \leftarrow (\neg L \vee R)$ ;
4     return toNNF( $n'$ ,  $neg$ );
5   else if  $n = (L \circ R)$  with  $\circ \in \{\wedge, \vee\}$  then
6      $\circ' \leftarrow \circ$ ;
7     if  $neg = \text{true}$  then
8        $\circ' \leftarrow \text{dual}(\circ)$ ;           //  $\text{dual}(\wedge) = \vee, \text{dual}(\vee) = \wedge$ 
9     end
10    return (toNNF( $L$ ,  $neg$ )  $\circ'$  toNNF( $R$ ,  $neg$ ));
11  else if  $n = (Qx. \phi)$  with  $Q \in \{\forall, \exists\}$  then
12     $Q' \leftarrow Q$ ;
13    if  $neg = \text{true}$  then
14       $Q' \leftarrow \text{dual}(Q)$ ;
15    end
16    return ( $Q'x.$  toNNF( $\phi$ ,  $neg$ ));
17  else if  $n = (\neg\phi)$  then
18    return toNNF( $\phi$ ,  $\neg neg$ );
19  else
20    // Atomic Leaf
21    if  $neg = \text{true}$  then return  $\neg n$ ;
22    else return  $n$ ;
23  end
```

4.3 The Probabilistic Budget Model

The generation of logically valid and structurally interesting formulas requires more than just adhering to fragment syntax; it requires a mechanism to steer the random generation toward specific user-defined shapes (e.g., forcing exactly 10 conjunctions and no existentials). The **FormulaBuilder** base class acts as the central engine for this probabilistic generation.

4.3.1 Constraint Management and Backtracking

The generation process is governed by the **BudgetState** structure, which keeps track of operational counters for each logical symbol (connectives and quantifiers) during a single tree-building session. The counters operate on a tri-state semantic:

- **Free** (-1): The symbol can be used infinitely.
- **Forbidden** (0): The symbol is completely excluded from the generation.
- **Constrained** ($n > 0$): The symbol must be used exactly n times. Each insertion decrements the counter.

Because generation is inherently stochastic, a randomly built branch might terminate prematurely (e.g., reaching maximum depth) before exhausting the constrained budget. To solve this, the generator wraps the building process in a backtracking loop: if a tree is completed but the `BudgetState` is not fully satisfied, the system throws a `BudgetRetryException`, discards the AST, and retries up to a `MAX_RETRY` limit of 500 attempts.

4.3.2 Applied Probability Distributions

The generation pipeline relies on a deterministic pseudo-random number generator (PRNG), initialized via a user-defined seed, to traverse the mathematical search space. The system’s randomness is abstracted by the `randInt(lo, hi)` utility, which wraps the C++ `std::uniform_int_distribution` to systematically model discrete probability distributions.

- **Uniform Selection** ($P(X = x) = 1/N$): When navigating the AST construction, the engine continuously selects the next logical operator or atomic predicate from a dynamic pool of valid candidates. A uniform distribution guarantees that, without external budget constraints, every structurally admissible symbol is equally likely to be instantiated.
- **Fair Bernoulli Trials** ($p = 0.5$): Binary decisions without inherent bias are modeled as fair coin tosses. For instance, when resolving a symmetric decision fork such as randomly selecting between a universal or existential quantifier when both are fully admissible, the system relies on a 50% probability (`randInt(0, 1) == 0`) to maintain a statistically balanced logic tree.
- **Weighted Bernoulli Trials and the 60% Heuristic** ($p = 0.60$): The system dynamically steers the random search space when strict budgets are applied. To maximize the success rate of the backtracking loop, the `pickType` method introduces a probabilistic bias. If there are “forced” symbols (those with a remaining budget $n > 0$), the generator selects from this restricted pool with an **60% probability**. This biased trial, computationally evaluated as `randInt(1, 100) ≤ 60`, gently guides the generation toward budget completion without collapsing into pure determinism.

4.4 Randomized Formula Generation

The core of the generation process is driven by the `FormulaBuilder` class, which recursively constructs Abstract Syntax Trees (ASTs) using the budget-constrained, randomized symbol selection strategy.

The formula generation procedure is formally outlined in Algorithm 2. The function `BUILD(depth, currentVar, budget)` takes three parameters: the remaining recursion depth (*depth*), the identifier of the active variable currently in

scope (*currentVar*), and the state tracking structure (*budget*) that monitors resource constraints or node allocation limits.

Algorithm 2: Recursive AST Generation via **FormulaBuilder**

```

1 function build(depth:  $\mathbb{N}$ , currentVar: String, budget: BudgetState):
  std::unique_ptr(ASTNode)
2   if depth = 0 then return buildAtomic(currentVar);
3   candidates  $\leftarrow$  candidateTypes(depth, budget);
4   if candidates.empty() then return buildAtomic(currentVar);
5   chosen  $\leftarrow$  pickType(depth, budget, candidates);
6   switch chosen do
7     case SymbolType::NEG do return
      make_unique(NegNode)(build(depth - 1, currentVar, budget)) ;
8     case SymbolType::AND do return
      make_unique(BinaryConnNode)(AND, build(...), build(...)) ;
9     case SymbolType::OR do return
      make_unique(BinaryConnNode)(OR, build(...), build(...)) ;
10    case SymbolType::IMPLIES do return
      make_unique(BinaryConnNode)(IMPLIES, build(...), build(...))
      ;
11    case SymbolType::IFF do return
      make_unique(BinaryConnNode)(IFF, build(...), build(...)) ;
12    case SymbolType::EXISTS do
13      boundVar  $\leftarrow$  nextVar(currentVar);
14      return
        make_unique(QuantifierNode)(EXISTS, Var(boundVar), ...
15        build(depth - 1, boundVar, budget));
16    end
17    case SymbolType::FORALL do
18      boundVar  $\leftarrow$  nextVar(currentVar);
19      return
        make_unique(QuantifierNode)(FORALL, Var(boundVar), ...
20        build(depth - 1, boundVar, budget));
21    end
22    case SymbolType::EQUALITY do
23      other  $\leftarrow$  nextVar(currentVar);
24      return
        make_unique(EqualityNode)(Var(currentVar), Var(other));
25    end
26    otherwise do return buildAtomic(currentVar) ;
27  end

```

The generation of formulas in the **Two-Variable Fragment** benefits directly from the simplicity of the base **FormulaBuilder**. Because the FO^2 restriction is purely quantitative (allowing at most two distinct variables in the

entire formula), it does not impose specific syntactic tree shapes or guard requirements. Therefore, the FO^2 generator directly inherits the standard recursive **build** algorithm (Algorithm 2), relying on a constrained variable allocator to guarantee that only two identifiers are ever issued during instantiation.

The **Fluted Fragment** restricts predicate arguments to continuous sequences of recently quantified variables. To enforce this, the **buildFL** algorithm replaces the simple current variable tracker with a *stack depth* indicator. The crucial difference lies in how variables are managed and forced into existence: if no admissible predicates can be formed at the current stack depth, the generator must force a quantification to extend the variable sequence.

Algorithm 3: Fluted Fragment: Stack-Depth and Generation

```

1 function buildFL(depth:  $\mathbb{N}$ , stackDepth:  $\mathbb{N}$ , budget: BudgetState)
2   admissible  $\leftarrow$  admissiblePreds(stackDepth);
3   if depth = 0  $\vee$  candidates(...).empty() then
4     if admissible  $\neq \emptyset$  then return buildAtomicFL(stackDepth);
5     return forcedQuant(depth, stackDepth);
6   end
7   // ... [Standard Boolean Connectives Omitted] ...
8   switch chosen do
9     case EXISTS or FORALL do
10      nextSD  $\leftarrow$  stackDepth + 1;
11      var  $\leftarrow$  varName(nextSD);
12      body  $\leftarrow$  buildFL(depth - 1, nextSD, budget);
13      return QuantifierNode(..., var, body);
14    end
15  end

```

The **Guarded Fragment** mandates that all quantified variables must be guarded by an atomic formula containing all free variables present in the quantification scope. Consequently, the **buildGF** generator must maintain a strict dynamic list of *current scope free variables*. When introducing a quantifier, it does not simply wrap the body; it actively generates a valid guard atom, binds a block of new variables, and joins them using the correct logical implication (for $\forall$) or conjunction (for $\exists$).

Algorithm 4: Guarded Fragment: Guarded Quantifier Generation

```
1 function buildGF(depth:  $\mathbb{N}$ , budget: BudgetState, Vfree: List)
  // ... [Standard Connectives Omitted] ...
2   switch chosen do
3     case EXISTS do
4       k  $\leftarrow$  pickValidArity(depth, Vfree);
5       Vbound  $\leftarrow$  nextVarNames(k);
6       G  $\leftarrow$  buildGuard(Vfree, Vbound);
7       body  $\leftarrow$  buildGF(depth - 1, budget, Vfree  $\cup$  Vbound);
8       return  $\exists V_{bound}. (G \wedge body)$ ;
9     end
10    case FORALL do
11      k  $\leftarrow$  pickValidArity(depth, Vfree);
12      Vbound  $\leftarrow$  nextVarNames(k);
13      G  $\leftarrow$  buildGuard(Vfree, Vbound);
14      body  $\leftarrow$  buildGF(depth - 1, budget, Vfree  $\cup$  Vbound);
15      return  $\forall V_{bound}. (G \implies body)$ ;
16    end
17  end
```

The **Unary Negation Fragment** imposes the critical restriction that negation can only be applied to formulas containing at most one free variable. To structurally guarantee this during randomized generation, **buildUN** deviates from the base builder in two ways. First, it completely overrides the standard **NEG** case, delegating it to a specialized **buildNegatedBody** routine that enforces the free-variable constraint. Second, if the current variable scope is empty and no valid predicates can be constructed, it triggers a forced quantification fallback to inject a new variable into the scope.

Algorithm 5: Unary Negation: Constrained Negation Generation

```
1 function buildUN(depth:  $\mathbb{N}$ , Vfree: List, budget: BudgetState)
2   admPreds  $\leftarrow$  admissiblePreds(Vfree);
3   if depth = 0  $\vee$  candidates(...).empty() then
4     if admPreds  $\neq \emptyset$  then return buildAtomicUN(Vfree);
5     return forcedQuant(depth, Vfree);
6   end
7   // ... [Standard Connectives Omitted] ...
8   switch chosen do
9     case NEG do
10      return buildNegatedBody(depth - 1, Vfree, budget);
11    end
12  end
```

To generate formulas representing **Modal Logic** problems, the **ModalGenerator**

applies the standard translation of modalities. It restricts quantifiers to simulate the modal operators $\Diamond$ (represented by **EXISTS**) and $\Box$ (represented by **FORALL**). Every quantified variable is strictly bound by an accessibility relation predicate, $r(x, y)$, connecting the current state to the next.

Algorithm 6: Modal Logic: Standard Translation Generation

```

1 function buildModal(depth:  $\mathbb{N}$ , currentVar: String, budget:
    BudgetState)
    // ... [Standard Connectives Omitted] ...
2   switch chosen do
3     case EXISTS ;                                // Simulates  $\Diamond\phi$ 
4     do
5       next  $\leftarrow$  nextVar(currentVar);
6       R  $\leftarrow$  r(currentVar, next) ;           // State accessibility
          relation
7        $\phi \leftarrow$  buildModal(depth - 1, next, budget);
          // Diamond requires conjunction
8       return  $\exists next. (R \wedge \phi)$ ;
9     end
10    case FORALL ;                                // Simulates  $\Box\phi$ 
11    do
12      next  $\leftarrow$  nextVar(currentVar);
13      R  $\leftarrow$  r(currentVar, next);
14       $\phi \leftarrow$  buildModal(depth - 1, next, budget);
          // Box requires implication
15      return  $\forall next. (R \implies \phi)$ ;
16    end
17  end

```

4.5 UNSAT Generation Algorithm

Generating unsatisfiable (UNSAT) formulas while strictly respecting a given structural budget presents a unique mathematical challenge. The framework employs a structural contradiction approach: generating a formula in the form $\Phi = \phi \wedge \neg\phi'$ (where ϕ' is an exact structural clone of ϕ but with distinct variables depending on the fragment rules).

To ensure the final AST (Φ) exactly matches the user's requested budget, the **FormulaBuilder** must mathematically divide the budget before recursively generating the sub-component ϕ . This requires the initial budget for **AND** and **NEG** to be odd (since the root connects the two branches with one **AND** and one **NEG**), while all other operators must be strictly even. The logic is formalized in Algorithm 7.

Algorithm 7: Budget-Constrained UNSAT Generation

```
1 function generateUNSAT( $d$ : Depth,  $B$ : Budget): Node
2   if  $B$ .AND is even  $\vee$   $B$ .NEG is even then throw Error;
3   if  $\exists x \notin \{\text{AND}, \text{NEG}\} \mid B.x$  is odd then throw Error;
4    $B$ .consume(AND);
5    $B$ .consume(NEG);
6    $B_\phi \leftarrow \{x \mapsto \lfloor B.x/2 \rfloor \mid x \in B\}$ ;
7    $\phi \leftarrow \text{buildComponentUNSAT}(d - 1, B_\phi)$ ;
8    $\phi' \leftarrow \text{clone}(\phi)$ ;
9   foreach  $x \in B$  do
10     $\Delta_x \leftarrow (B.x/2) - B_\phi.x$ ;           // Actual nodes used by  $\phi$ 
11     $B.x \leftarrow \max(0, B.x - (\Delta_x \times 2))$ ;
12  end
13  return  $(\phi \wedge \neg \phi')$ ;
```

To enforce the specific syntactic constraints of each logical fragment, the base `generateUNSAT` method relies on dynamic dispatch. The construction of the sub-formula ϕ is delegated to a polymorphic `buildComponentUNSAT` method, which is uniquely implemented for each fragment (e.g., managing variable stacks in Fluted logic, or prepending strict quantifiers in FO^2).

4.6 Fragment-Specific Methods

For most fragments, generating a satisfiable (SAT) formula relies on a rejection-sampling approach: formulas are generated freely within the syntactic constraints, and an external theorem prover (like Vampire) is invoked to filter out the unsatisfiable ones.

However, for the Two-Variable Fragment (FO^2), the system implements a highly optimized, *correct-by-construction* SAT generation engine.

4.6.1 The Two-Variable Fragment (FO^2) and SAT Construction

The standard FO^2 generation inherently guarantees the two-variable restriction by toggling a single active variable state ($V_1 \leftrightarrow V_2$) during the recursive descent, thus eliminating the need for complex variable binding environments.

To natively generate SAT instances without relying on external provers, the system introduces the `FO2SATGenerator`. This generator shifts the paradigm from purely syntactic generation to *model-driven generation*.

Model-Driven Target Propagation

The core idea is to first instantiate a random `FiniteModel` over a small domain and then recursively build an AST that evaluates to `true` under a specific set of target assignments (`Targets`). To verify truth values during generation, the

system employs the **F02Evaluator**, a specialized visitor that evaluates AST nodes against the finite model using standard logical semantics.

As the tree is built downward, the **Targets** are algebraically propagated and manipulated depending on the logical connective chosen, as formalized in Algorithm 8.

Algorithm 8: Target Propagation for FO² SAT Generation

```

1 function buildSAT(d: Depth, T: Targets, M: Model, v: Var): Node
2   if d = 0 then return buildAtomicSAT(T, M, v);
3   op ← pickType(d);
4   if op = AND then
5     return (buildSAT(d − 1, T) ∧ buildSAT(d − 1, T));
6   else if op = OR then
7     (TL, TR) ← split(T);
8     return (buildSAT(d − 1, TL) ∨ buildSAT(d − 1, TR));
9   else if op = IMPLIES then
10    (TL, TR) ← split(T);
11    TA ← complementTargets(M, TL, v);
12    return (buildSAT(d − 1, TA) → buildSAT(d − 1, TR));
13  else if op = NEG then
14    TC ← complementTargets(M, T, v);
15    return ¬buildSAT(d − 1, TC);
16  else if op = EQUALITY then
17    return buildEqualitySAT(T, M, v);
18  else if op ∈ {∃, ∀} then
19    T' ← expandTargets(T, M, op);
20    return (Qv. buildSAT(d − 1, T'));

```

When descending into quantified nodes (∃, ∀), the bound variables must be dynamically updated in the target list. The **expandTargets** subroutine (Algorithm 9) manages this by either randomizing the bound variable assignment (for ∃) or exhaustively expanding it across the entire domain (for ∀). The routine aggressively caps the maximum number of targets at 1000 to maintain constant-time bounds during deep recursions.

Algorithm 9: Quantifier Target Expansion (expandTargets)

```
1 function expandTargets( $T, M, op$ )
2    $T' \leftarrow \emptyset$ ;
3    $D \leftarrow M.\text{domainSize}()$ ;
4    $v_b \leftarrow 1 - v$  ;                                // Bound var
5   foreach  $t \in T$  do
6     if  $|T'| \geq 1000$  then break;
7     if  $op = \exists$  then
8        $ex \leftarrow t$ ;
9        $ex[v_b] \leftarrow \text{randInt}(0, D - 1)$ ;
10       $T' \leftarrow T' \cup \{ex\}$ ;
11    else if  $op = \forall$  then
12      foreach  $e \in [0, D - 1]$  do
13        if  $|T'| \geq 1000$  then break;
14         $ex \leftarrow t$ ;
15         $ex[v_b] \leftarrow e$ ;
16         $T' \leftarrow T' \cup \{ex\}$ ;
17      end
18    end
19  return  $T'$ ;
```

Sparse Complement Optimization

A significant bottleneck in target propagation arises during negation (NEG) and the antecedent of implication (IMPLIES), which require computing the complement of the current targets within the domain space. A naive multi-dimensional array representation for domain assignments would result in severe memory explosion for larger domains.

To overcome this, the `complementTargets` procedure utilizes a sparse memory approach. Instead of allocating the full Cartesian product, the algorithm hashes the active assignments into a 1D unique key (e.g., $e_1 \times |D| + e_2$) and stores them in an `unordered.set`. The complement is then rapidly generated by querying this hash map and capped at a `MAX_TARGETS` limit of 1000 to maintain constant-time bounds during deep recursions.

Leaf Resolution

At the bottom of the tree ($d = 0$), `buildAtomicSAT` serves as the anchor point. It iterates through the randomized vocabulary and checks the `FiniteModel` to find a predicate that is uniformly `true` (or uniformly `false`) for *all* assignments in the current `Targets`. If it finds a uniformly true predicate, it returns the atomic node; if uniformly false, it wraps the atom in a `NegNode`. This ensures the constructed formula perfectly matches the propagated truth requirements of the root model.

Engineering Optimizations

For the FO^2 fragment, the SAT formula generation is coupled to an explicit finite model $\mathcal{M} = \langle D, I \rangle$. The domain size follows a purely exponential formula: $|D| = 2^{N \times (|\text{unary}| + |\text{binary}|)}$. To support executions on massive domains without triggering Out-Of-Memory (OOM) errors, the architecture was deeply optimized:

- **Relation Compression (Bit-Vectors):** The quadratic overhead $O(|D|^2)$ of storing binary relations was resolved by “flattening” two-dimensional matrices into 1D bit vectors (`std::vector<bool>`), allowing highly optimized $O(1)$ access via linearized indexing ($e_1 \times |D| + e_2$).
- **Hash-Based Search for the Complement:** The `complementTargets` function converts the target vector into an `std::unordered_set` representing a 1D hashed key. This shifts the lookup mechanism from a linear or logarithmic scan to a hash-based query, reducing the complement calculation overhead significantly during generation.
- **Zero-Allocation Parsing & Scope Hoisting:** The `fastParseIndex` function avoids heap allocations by converting strings to integers via pointer arithmetic. Superfluous map cloning inside evaluation loops was eliminated by instantiating maps outside the loop, maximizing CPU cache locality.
- **Integer Overflow Prevention:** The framework explicitly caps the total exponent calculation at 13, restricting the maximum domain to 8,192 elements. Exceeding this limit (e.g., reaching 65,536) would push binary predicate matrices over 4.2 billion cells, surpassing the 32-bit signed `INT_MAX` threshold and causing critical integer overflows during indexing.

4.6.2 The Fluted Fragment (FL): Forced Quantification

A unique challenge in randomly generating $\mathcal{FL}$ formulas arises when the generation budget dictates the creation of a leaf node (i.e., `depth == 0`), but the active vocabulary lacks predicates that fit the current `stackDepth`. Specifically, the `admissiblePreds` function filters out any predicate whose arity exceeds `stackDepth`.

To prevent generation deadlocks, the algorithm introduces a `forcedQuant` fallback closure. If no admissible predicates are available, the generator forcefully consumes an available quantifier from the budget, increments the `stackDepth`, and recursively attempts to build the body. This mechanism guarantees that the generation process will “dig” deep enough to eventually satisfy the arity requirements of the vocabulary, as outlined in Algorithm 10.

Algorithm 10: $\mathcal{FL}$ Forced Quantification

```
1 function buildFL(d: Depth, s: StackDepth, B: Budget): Node
2   A  $\leftarrow$  admissiblePreds(s)
3   if d = 0  $\vee$  candidatesEmpty() then
4     if A  $\neq \emptyset$  then
5       return buildAtomicFL(s);
6     else
7       if B.EXISTS = 0  $\wedge$  B.FORALL = 0 then throw
8         BudgetRetryException;
9       Q  $\leftarrow$  pickAvailableQuantifier(B) ;
10      B.consume(Q);
11      xnew  $\leftarrow$  varName(s + 1);
12      body  $\leftarrow$  buildFL(max(0, d - 1), s + 1, B);
13      return (Qxnew.body);
14    end
15  ...// Standard recursive generation for connectives
```

4.6.3 The Guarded Fragment (GF): Guard Generation

The Guarded Fragment restricts quantification by mandating that any quantified sub-formula must be shielded by an atomic formula (the guard) containing all free variables in the current scope. To enforce this, the **GuardedGenerator** actively maintains a state vector, **currScopeFreeVars**, representing the variables $\bar{x}$ that are free in the current recursive step.

On-the-fly Guard Synthesis

When the generation algorithm selects a quantifier (e.g., an existential node), it cannot simply bind new variables $\bar{y}$. It must simultaneously guarantee the existence of a valid guard predicate $\alpha(\bar{x}, \bar{y})$ whose arity is exactly equal to the total number of free and newly bound variables ($|\bar{x}| + |\bar{y}|$).

The system queries the active vocabulary via the **admissibleGuards** method to find matching predicates [cite: 25]. If no such predicate exists, the quantifier choice is dynamically rejected via a **BudgetRetryException**. If a guard is found, the system updates the scope, builds the body, and wraps the structural components logically (using $\wedge$ for $\exists$, and $\rightarrow$ for $\forall$), as formalized in Algorithm 11.

Algorithm 11: GF Existential Guard Generation

```
1 function buildGuardedExists(d: Depth, B: Budget,  $\bar{x}$ : ScopeVars):  
   Node  
2    $k_{\max} \leftarrow \min(3, d)$  ;  
3    $V_k \leftarrow \{k \in [1, k_{\max}] \mid \text{admissibleGuards}(|\bar{x}| + k) \neq \emptyset\}$  ;  
4   if  $V_k = \emptyset$  then throw BudgetRetryException;  
5    $k \leftarrow \text{pickRandom}(V_k)$ ;  
6    $\bar{y} \leftarrow \text{nextVarNames}(k)$ ;  
7    $\alpha \leftarrow \text{buildGuard}(\bar{x}, \bar{y})$ ;  
8    $\text{body} \leftarrow \text{buildGF}(d - 1, B, \bar{x} \cup \bar{y})$ ;  
9   return  $(\exists \bar{y}. (\alpha \wedge \text{body}))$ ;
```

4.6.4 The Unary Negation Fragment (UNFO): Truncation

The Unary Negation Fragment (UNFO) allows full First-Order Logic syntax with one critical restriction: the negation operator ($\neg\phi$) can only be applied to sub-formulas ϕ that contain at most one free variable. The `UnaryNegGenerator` handles this elegantly without requiring complex dependency graphs.

Negation Scope Reduction

Similar to the Guarded Fragment, the UNFO generator tracks active variables via a `currFreeVars` vector. However, the crucial intervention occurs when the random builder selects a `NEG` node. Instead of passing the entire active scope to the child node, the `buildNegatedBody` method deliberately truncates the scope. It randomly selects a single variable from `currFreeVars` and passes only that variable downward, effectively enforcing the UNFO rule by construction.

Algorithm 12: UNFO Negation Scope Truncation

```
1 function buildNegatedBody(d: Depth, B: Budget,  $\bar{x}$ : ScopeVars):  
   Node  
2    $V_{\text{unary}} \leftarrow \emptyset$ ;  
3   if  $\bar{x} \neq \emptyset$  then  
4      $v \leftarrow \text{pickRandom}(\bar{x})$ ;  
5      $V_{\text{unary}} \leftarrow \{v\}$ ;  
6   end  
7    $\text{body} \leftarrow \text{buildUN}(d - 1, B, V_{\text{unary}})$  ;  
8   return  $\neg \text{body}$ ;
```

Scope Injection via Forced Quantification

A secondary consequence of the UNFO syntax is that a formula might reach a leaf node with an empty `currFreeVars` state (e.g., at the very root of the AST before any quantifiers are generated). If the active vocabulary only contains predicates of arity ≥ 1 , generating a valid atom becomes impossible. To resolve

this, the system employs a `forcedQuant` heuristic. If no variables are in scope, it bypasses the standard connective randomizer, forcefully consumes a quantifier from the budget, and recursively injects a new variable into the scope, ensuring that atomic leaves can always be populated correctly.

4.7 Output Serialization and TPTP Formatting

Once the generation constraints are met and the Abstract Syntax Tree is fully constructed in memory, the system must serialize the logical formula into a machine-readable text format. While the AST supports standard human-readable stringification via the `toString()` method, the primary target format is the TPTP (Thousands of Problems for Theorem Provers) syntax. This format acts as the *lingua franca* for modern automated theorem provers such as Vampire, E, and SPASS, making it essential for the evaluation pipeline.

4.7.1 Polymorphic Translation and Lexical Sanitization

Serialization is handled polymorphically through the `toTPTP()` virtual method, implemented by every concrete node in the AST hierarchy. During the recursive traversal of the tree, the internal C++ symbolic representations are mapped to TPTP-compliant ASCII operators:

- **Operators:** Logical AND ($\wedge$), OR ($\vee$), IMPLIES ($\rightarrow$), and NEG ($\neg$) are translated to `&`, `|`, `=>`, and `~` respectively. Universal ($\forall$) and existential ($\exists$) quantifiers are translated using `!` and `?`.

Simultaneously, the AST enforces strict TPTP lexical constraints: variables must begin with an uppercase letter, whereas predicates must begin with a lowercase letter. Because different fragments use varying variable naming conventions (e.g., `v1`, `x1`, `w1`), the nodes perform dynamic lexical sanitization. When `AtomicNode::toTPTP()` or `QuantifierNode::toTPTP()` are invoked, predicate names are systematically downcased via standard library utilities, and every variable is processed by the static `tptpVar()` method, which capitalizes the first character. This built-in sanitization ensures that the generated output remains syntactically valid regardless of the internal fragment’s vocabulary.

4.7.2 Semantic Roles

The final step of serialization occurs at the root of the `FormulaBuilder`, which encapsulates the generated TPTP string into the standard First-Order Form wrapper structure: `fof(f, role, formula)..`

Crucially, the orchestrator dynamically assigns the semantic *role* based on the requested generation mode:

- **axiom:** Assigned during `FREE` and `SAT` generation modes. This instructs the external theorem prover to treat the generated formula as a given premise in its knowledge base.

- **negated_conjecture:** Assigned during **UNSAT** mode. Because UNSAT generation produces structurally contradictory formulas $(\phi \wedge \neg \phi')$, tagging them as negated conjectures aligns the synthetic output with standard refutational theorem-proving workflows, optimizing the provers' internal parsing heuristics.

Chapter 5

Vampire Theorem Prover

To effectively verify the satisfiability of logically generated fragments, this research relies on **Vampire** [10], a state-of-the-art Automated Theorem Prover (ATP). This chapter outlines the specific theoretical mechanisms Vampire employs to process First-Order Logic (FOL) formulae, execute heuristic searches, and determine satisfiability.

5.1 First-Order Clausification

While modern ATPs accept standard First-Order Form (**fof**) syntax, their core reasoning engines require formulae to be strictly structured in Conjunctive Normal Form (CNF). When Vampire receives a **fof** input, it immediately applies an equisatisfiable clausification pipeline [10]:

- **Negation Normal Form:** Implications are expanded and negations are pushed inward.
- **Skolemization:** Existential quantifiers are substituted with Skolem functions.
- **CNF Conversion:** The formula is flattened into a set of clauses (disjunctions of literals).

5.2 The Superposition Calculus

Vampire operates via a saturation-based refutation loop driven by the *Superposition Calculus* [2]. This calculus extends standard resolution by inherently handling equality (=) through strict term-reduction orderings (e.g., Knuth-Bendix Ordering). By treating equations as directional rewrite rules (from complex to simple terms), superposition drastically trims the search space, preventing the infinite loops typical of naive equality substitution.

5.3 Saturation Loop and Given Clause Algorithm

To systematically derive logical contradictions or establish saturation, Vampire organizes its core proof search around the *Given Clause algorithm*. This mechanism operates on a search space partitioned into distinct clause sets, ensuring fairness and completeness through rigorous clause management:

- **Unprocessed Clauses:** Newly generated or input clauses awaiting initial simplification and redundancy checks.
- **Passive Clauses:** Clauses that have passed preliminary screening but have yet to participate fully in generating new inferences. Clause selection heuristics dictate which passive clauses enter the active set next.
- **Active Clauses:** Clauses that have been fully processed, meaning all pairwise generating inferences between them and other active clauses have already been executed.

The foundational saturation loop updates these sets iteratively:

1. Non-redundant unprocessed clauses are selected, subjected to forward simplification using processed clauses, and moved to the passive set.
2. A single *given clause* is selected from the passive set using sophisticated queue-weighting heuristics and transferred to the active set.
3. All possible generating inferences (such as superposition and resolution) are performed between this new given clause and all existing active clauses.
4. Newly inferred clauses are fed back into the unprocessed pool after redundancy elimination (including subsumption and tautology deletion).

When this loop runs to completion without generating an empty clause, the resulting active set represents a true *saturated* state, confirming the satisfiability of the original fragment.

5.4 Portfolio Solving and CASC Mode

Due to the semi-decidable nature of FOL, a single proof strategy is rarely effective across diverse logical fragments. To address this, Vampire implements *Portfolio Solving* [10]. When operating in CASC mode (named after the CADE ATP System Competition), Vampire does not rely on a single heuristic. Instead, it sequentially or concurrently executes a scheduled portfolio of distinct saturation strategies, time-slicing the available execution window to maximize the probability of proof discovery.

5.5 Satisfiability and Finite Model Finding

While traditional ATPs excel at proving unsatisfiability (UNSAT) by deriving an empty clause, evaluating decidable fragments often requires confirming satisfiability (SAT). Vampire achieves this through two primary theoretical mechanisms:

- **Saturation Termination:** If the Given Clause algorithm exhausts all possible inferences without generating an empty clause, the clause set is fundamentally satisfiable.
- **Finite Model Finding (FMF):** Vampire employs specialized SAT/SMT-backed engines to actively search for boolean models over finite domains [15].

To efficiently manage complex formulas and model finding, modern versions of Vampire rely on the AVATAR architecture (Advanced Vampire Architecture for Theories and Resolution) [15]. As illustrated in Figure 5.1, AVATAR splits the reasoning process: a dedicated SAT solver handles the propositional boolean structure, while the First-Order engine handles the superposition calculus. If the SAT solver finds a valid boolean model, the prover can successfully halt and classify the underlying formula as SAT.

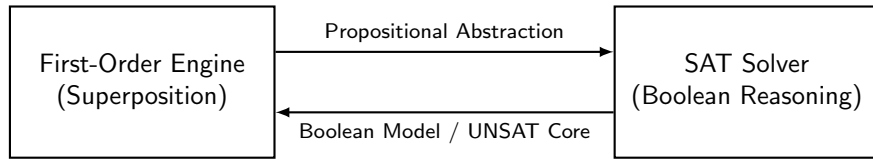


Figure 5.1: Conceptual diagram of the AVATAR architecture adapted from [15]. The workload is split between a standard superposition engine and an embedded SAT solver, exchanging abstractions and models to efficiently determine satisfiability.

Chapter 6

Testing and Experimental Validation

6.1 The Benchmarking Framework

This chapter illustrates the experimental validation methodology adopted to evaluate the effectiveness, efficiency, and structural limits of the developed logic generators across all five supported fragments (Fluted, FO^2 , Guarded, UNFO, and Modal). Testing was conducted through a custom, fully automated **Benchmarking Framework**

This framework manages the entire end-to-end experimental pipeline:

1. **Parametric Generation:** Recursively scanning dataset directories for TPTP files, calculating formula lengths, and extracting internal generation metadata (such as the number of generation attempts) directly from file headers via regular expressions (`% ATTEMPTS: <count>`).
2. **Oracle Validation:** Automatically feeding the formulas to the Vampire theorem prover acting as an oracle under configurable time constraints and saturation modes.
3. **Sorting, Reporting, and Calibration:** Parsing Vampire’s standard output (SZS status), capturing runtime metrics and automatically routing problematic formulas—such as Timeouts, Errors, or Unknowns into isolated subdirectories (e.g., `timeout_cand<t>` and `unknown_cand<t>`) for timeout calibration. All execution results are listed into structured CSV reports containing comprehensive metrics.

6.2 Execution and Timeout Calibration

To ensure rigorous evaluation the framework executes Vampire in his saturation mode. An autonomous routing mechanism relocates formulas trigger-

ing **Timeout**, **Error**, or **Unknown** statuses to dedicated directories. For the Modal Fragment, evaluation under pure saturation systematically stalls formulas into uniform timeouts; thus, execution shifted to Vampire’s strategy-scheduling CASC mode. CASC dynamically sequences multiple strategies to unlock the modal search space.

6.3 Dataset Generation and Analysis

Our dataset generation adopts an **Isolated Scaling** strategy. By systematically isolating structural dimensions across different logical fragments, we can precisely measure how theorem provers react to specific theoretical barriers. Every configuration is instantiated in both satisfiable (**sat**) and unsatisfiable (**unsat**) modes to evaluate refutational completeness alongside model-finding capabilities.

6.3.1 FO^2 Fragment Datasets

Dataset generation for the two-variable fragment (FO^2) is partitioned into six dimensions:

- **Vocabulary Scaling:** Progressively expanding distinct predicates from 5/1, 2/2 up to 20/1, 10/2 at a fixed depth of 12.
- **Depth Scaling:** Syntax tree depths scaled across $d \in \{5, 8, 12, 16\}$ using a standard vocabulary (15/1, 10/2).
- **Quantifier Scaling:** Incrementing quantifier budgets $qb \in \{2, 4, 6, 8\}$, with depth dynamically adjusted as $qb \times 2 + 10$.
- **Equality Scaling:** Progressively increasing equality operator budgets $eq \in \{2, 4, 6, 8\}$ at depth 18 with a specialized vocabulary (5/1, 15/2).
- **Finite Domain Scaling (SATBUILD):** Targeting finite model generation robustness by explicitly scaling the explicit domain size $ds \in \{1, 2, 3\}$ at depth 12 with a minimal vocabulary (2/1, 1/2).
- **Dense Equality & IFF:** Stress-testing heavy equality and equivalence interactions with dense budgets $eq \in \{10, 16, 20, 26, 30\}$ at depth 30 using vocabulary 2/1, 2/2.

Figure 6.1 maps the distribution of Vampire’s SZS statuses across each FO^2 dataset category, providing a view of the solver’s performance and its failure modalities under different structural stresses.

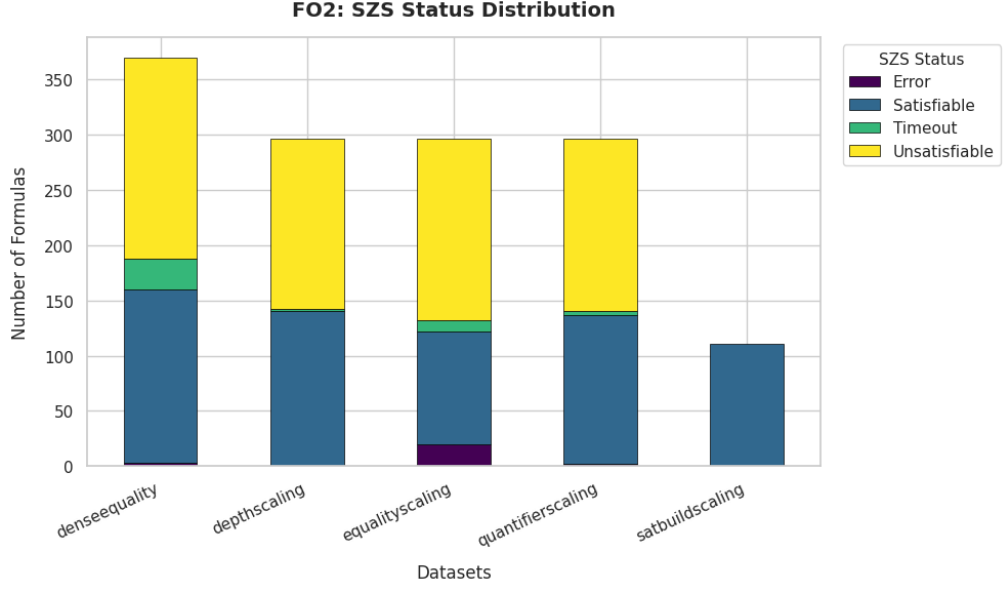


Figure 6.1: SZS status distribution across FO^2 scaling datasets.

As Figure 6.1 illustrates, while standard scaling dimensions are handled smoothly, heavy equality interactions fracture into two distinct failure modes: *Timeouts* for `denseequality` and out-of-memory *Errors* for `equalityscaling`.

This contrast is caused by the relational vocabulary. Although `denseequality` features much deeper formulas (depth 30), it relies on just two binary predicates. In contrast, `equalityscaling` deploys 15. Within Vampire’s superposition calculus, pairing a broad binary vocabulary with equality triggers a severe combinatorial explosion. Consequently, this relational density overwhelms the search space, causing the solver to abruptly crash due to memory exhaustion well before reaching the timeout.

6.3.2 Fluted Fragment Datasets

For Fluted Logic, scaling isolates four core structural axes:

- **Depth Scaling:** Varying the maximum syntactic tree depth $d \in \{6, 10, 14, 18, 22\}$ using a standard vocabulary of unary, binary, and ternary predicates (10/1, 5/2, 2/3).
- **Quantifier Scaling:** Incrementing universal and existential quantifier budgets $qb \in \{4, 8, 12, 16\}$ at a fixed depth of 18, supported by a richer vocabulary (10/2, 5/3, 3/4).
- **Branching Scaling:** Scaling conjunction and disjunction budgets $bb \in \{6, 10, 14, 18\}$ at depth 14 with a propositional vocabulary (20/1, 10/2).

- **Arity & IFF Cross-Scaling:** Testing deeper dependencies with predicate arities set to 6/3 and 3/5, combined with equivalence budgets ($iff_val \in \{4, 6\}$) at depth 22.

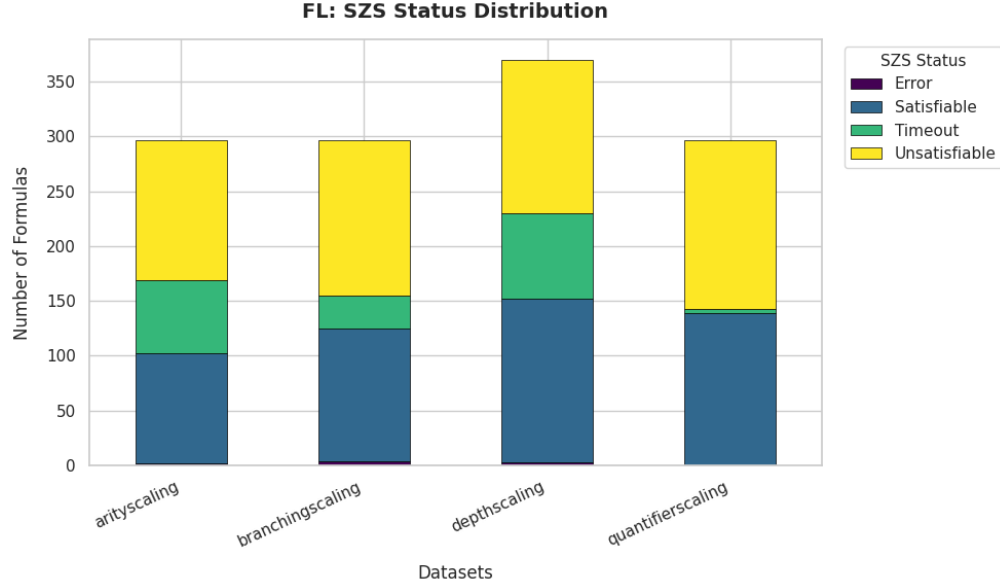


Figure 6.2: SZS status distribution across Fluted logic isolated scaling datasets.

As illustrated in Figure 6.2, the empirical evaluation via Vampire demonstrates that structural complexity in Fluted Logic is heavily dictated by predicate arity and depth. Specifically, **arityscaling** and **depthscaling** concentrate the highest density of timeouts due to the exponential expansion of the superposition search space, whereas **quantifierscaling** remains with near-zero failures.

6.3.3 Guarded Fragment Datasets

Targeting guarded logics using a base vocabulary (20/1, 10/2, 5/3), the generation explores:

- **Depth Scaling:** Tree depths ranging from $d \in \{10, 18, 26, 34\}$, where quantifier and connective budgets scale proportionally to depth (e.g., existential/universal budgets set to $\{d/4, d/2\}$).
- **Quantifier Alternation Scaling:** Incrementing alternation budgets $qb \in \{4, 8, 12, 16\}$ at a fixed depth of 22.

- **Branching Scaling:** Massive connective budgets $bb \in \{20, 35, 50, 65\}$ at depth 30 with an expanded vocabulary (40/1, 20/2, 10/3) covering AND, OR, IMPLIES, and IFF.
- **Equality Scaling:** Equality counts scaled across $eq \in \{4, 8, 12, 16\}$ at depth 18.

Figure 6.3 SZS status distribution across Guarded logic isolated scaling datasets.

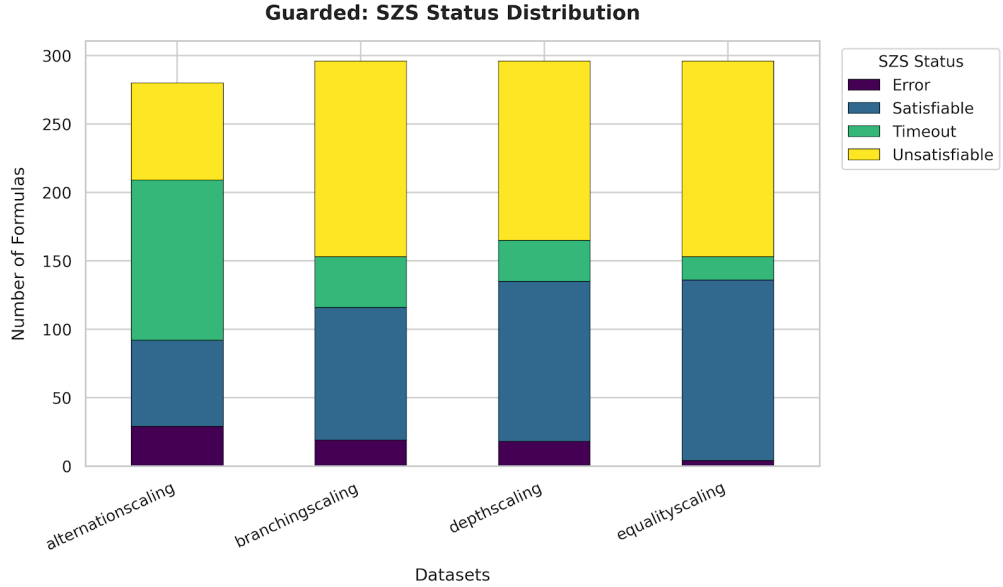


Figure 6.3: SZS status distribution across Guarded Fragment scaling datasets.

As Figure 6.3 illustrates, the Guarded Fragment exhibits a pronounced sensitivity to quantifier alternation. The `alternationscaling` dataset triggers the highest concentration of both *Timeouts* and out-of-memory *Errors*. In guarded logic, deep quantifier alternations enforce complex guard predicate dependencies. During clausification, this structural nesting forces the solver to generate high-arity functions (Skolem), that rapidly overwhelms Vampire’s memory allocation.

On the other hand, `equalityscaling` is processed with efficiency, yielding the highest rate of definitive resolutions (SAT/UNSAT). Because its quantifier budgets are strictly bounded, the term space remains compact, this allows Vampire’s equality rules to simplify the search space, resolving the structural constraints.

6.3.4 Unary Negation Fragment Datasets

For the unary negation fragment, using a standard vocabulary (15/1, 10/2, 5/3), parameters are modulated by:

- **Depth Scaling:** Structural tree depths evaluated at $d \in \{6, 11, 16, 21\}$.
- **Quantifier Alternation Scaling:** Quantifier budgets scaled across $qb \in \{2, 5, 8, 11\}$ at depth 18.
- **Negation Scaling:** Explicitly increasing negation frequency via `not_count` budgets $nb \in \{6, 10, 14, 18\}$ at depth 20 using vocabulary 10/1, 5/2.
- **Equality Scaling:** Equality budgets scaled from $eq \in \{2, 4, 6, 8\}$ at depth 16.

Figure 6.4 maps the SZS statuses for UNFO, highlighting a distinct shift in how the solver handles structural stress compared to other fragments.

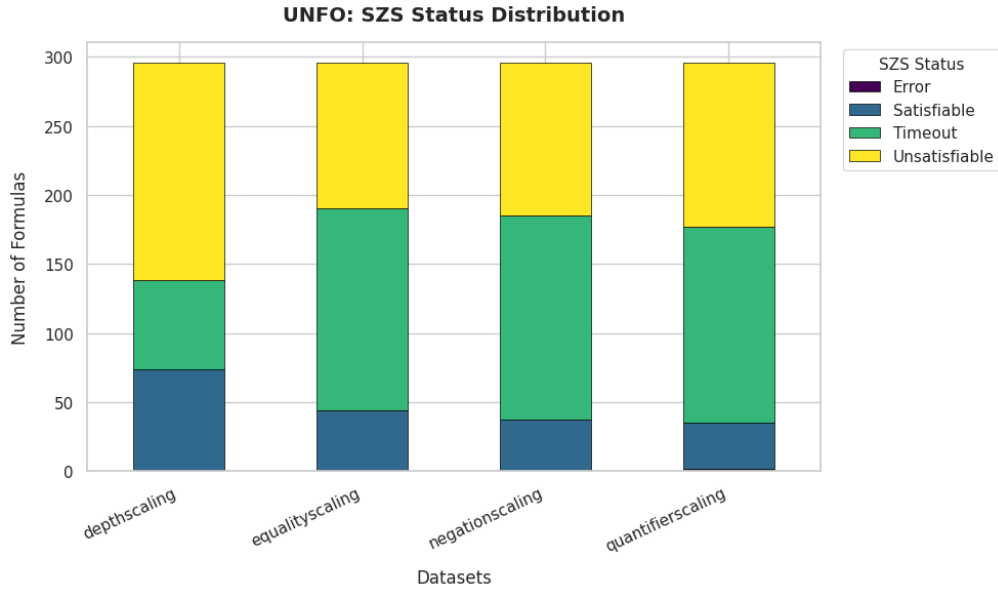


Figure 6.4: SZS status distribution across UNFO scaling datasets.

Unlike the Guarded and FO^2 fragments, UNFO datasets completely avoid memory-based *Errors*, indicating highly stable memory consumption. However, the solver incurs a massive rate of *Timeouts* when scaling equalities, negations, and quantifiers. Only **depthscaling** maintains a robust completion rate. This reveals that in UNFO, pure structural depth is easily navigated, whereas negations and equalities traps the solver in non-terminating search paths that exhaust time well before exhausting RAM.

6.3.5 Modal Fragment Datasets

Finally, modal logic datasets isolate constraints over modal structures:

- **Modal Depth Scaling:** Nesting depths ranging from $d \in \{4, 8, 12, 16, 20\}$ using a propositional vocabulary (15/1).
- **Box-Diamond Alternation Scaling:** Modulating modal operator interactions with alternation budgets $qb \in \{2, 4, 6, 8, 10\}$ at depth 12.
- **Propositional Branching Scaling:** Branching budgets scaled across $bb \in \{4, 8, 12, 16, 20\}$ at depth 10 using vocabulary 20/1.
- **Vocabulary Scaling:** Expanding the propositional variable pool from 5/1 up to 50/1 at a fixed depth of 10.

Figure 6.5 visualizes Vampire’s performance distribution across these modal logic generation axes.

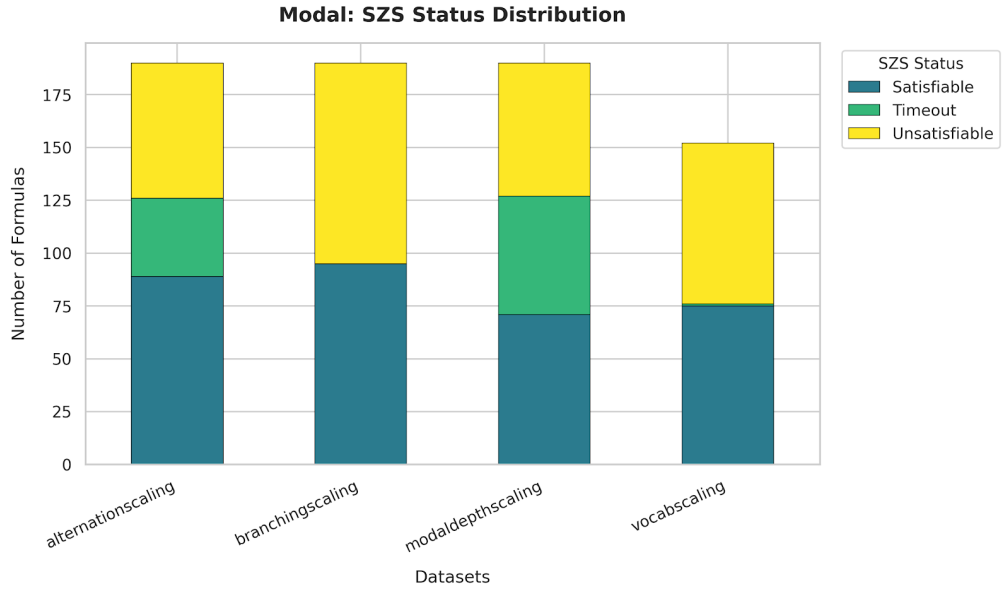


Figure 6.5: SZS status distribution across Modal logic scaling datasets.

As Figure 6.5 illustrates, the Modal Fragment features a complete absence of out-of-memory *Errors*. Within the generator, modal operators are mapped via standard relational translation to **FORALL** and **EXISTS** quantifiers bound by a single binary accessibility relation **r**, while the underlying vocabulary is strictly restricted to unary predicates. This structural constraint inherently prevents

the massive high-arity Skolemization that crashes the solver in the Guarded Fragment, keeping memory consumption stable.

Crucially, obtaining these resolved outcomes required executing Vampire under its CASC mode (automatic strategy scheduling) rather than saturation mode. Under pure saturation, modal formulas systematically stall into uniform timeouts; CASC’s scheduling successfully unlocks the search space, handling the distribution of satisfiable and unsatisfiable statuses shown in Figure 6.5.

Nonetheless, structural stress still manifests as *Timeouts*, heavily concentrated in **modaldepthscaling** and **alternationscaling**. Deeply nested boxes and diamonds generate long, alternating chains of quantifiers over the accessibility relation **r**. Evaluating these deep relational paths forces the solver into exhaustive searches, whereas expanding propositional width (**branchingscaling** and **vocabscaling**) is handled effortlessly by Vampire’s optimized propositional reasoning.

6.4 Resolution Times and Complexity Analysis

To evaluate how syntactic size impacts theorem-proving performance, Figures 6.6, 6.7, 6.8, and 6.9 map Vampire’s resolution times against formula length between all four first-order fragments under saturation mode.

A prominent idea is that **formula length is not proportional to computational complexity**. Syntactically compact formulas (ranging between 10^2 and 10^4 characters) frequently trigger exhaustive searches that culminate in 30s or 60s *Timeouts*. Contrarily, larger formulas—spanning up to 10^5 or even 10^7 characters are frequently resolved successfully in few seconds. This highlights that automated deduction efficiency is dictated by internal structural composition (such as quantifier nesting, equality density, and term depth) rather than string length.

Timeouts do not scale monotonically with size; instead, they concentrate in specific intermediate length bands. In these regions, the generated clause sets maximize the search space for Vampire’s superposition calculus without providing the direct refutations or saturation points required for quick termination.

Fragment-Specific Trajectories:

- **FO^2 and Fluted Fragments:** Demonstrate localized timeout clusters, where the solver either resolves formulas swiftly or hits the hard temporal ceiling abruptly.
- **Guarded Fragment:** Exhibits severe timeout density at moderate lengths (2×10^3 to 10^5), driven by high-arity Skolemization constraints.
- **Unary Negation Fragment (UNFO):** Displays a distinct correlation for successfully resolved UNSAT instances at extreme lengths (10^6 to 10^7 characters), reflecting a more predictable computational trajectory.

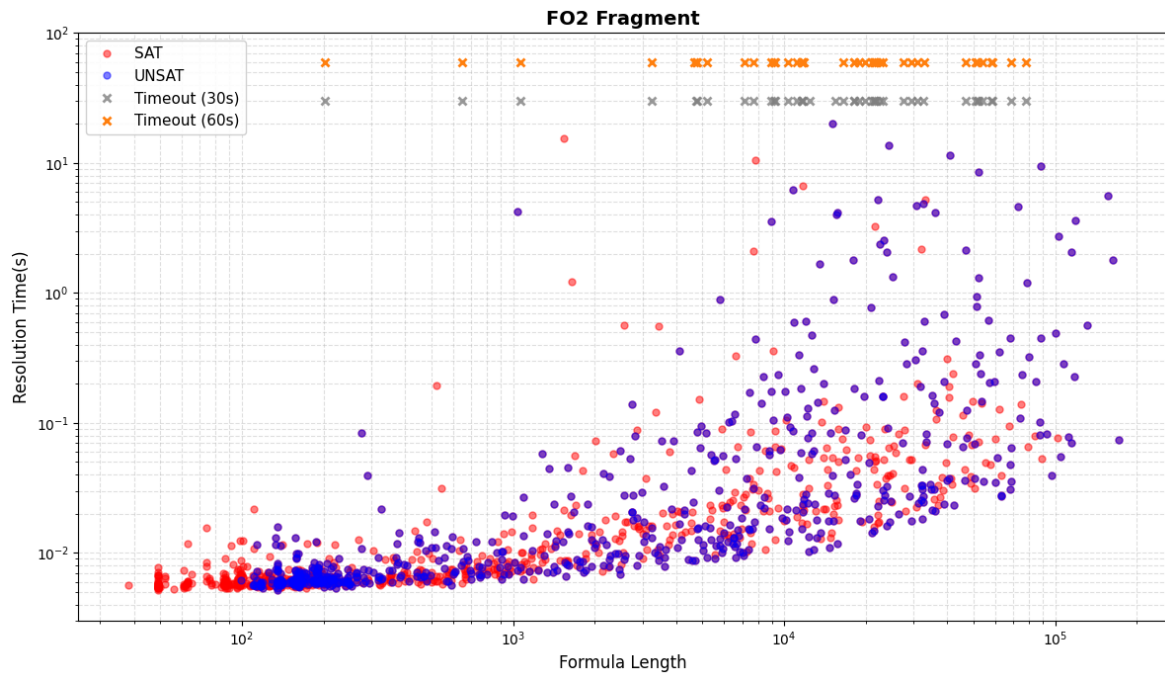


Figure 6.6: Resolution time vs. formula length for the FO^2 fragment.

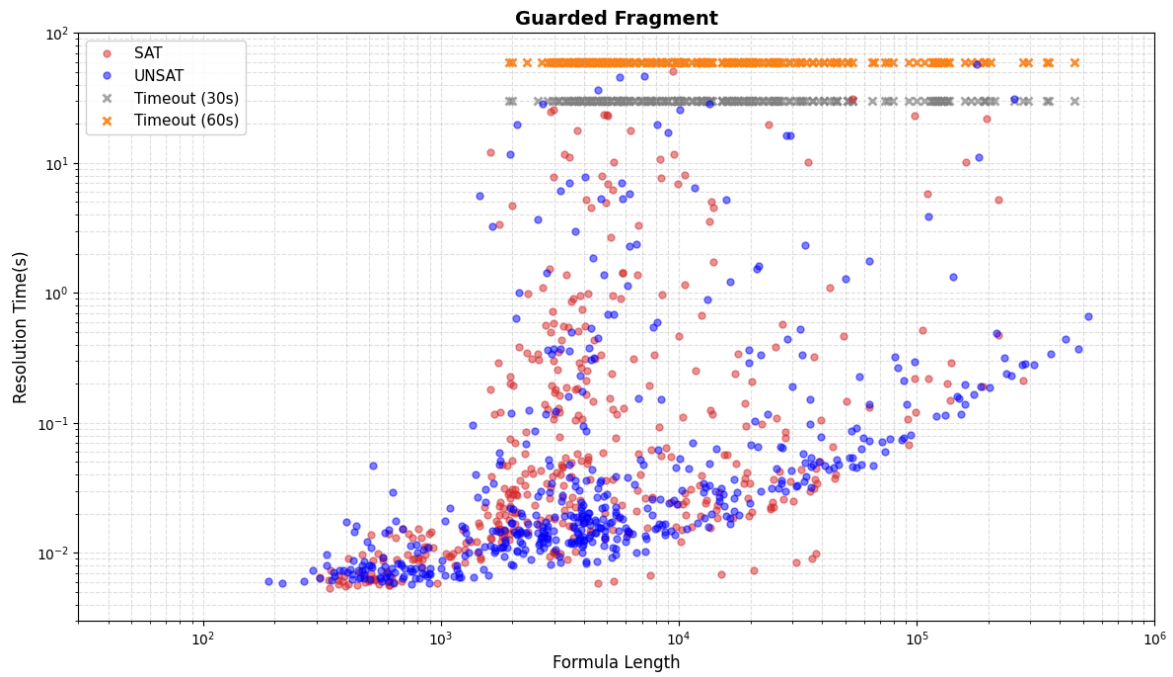


Figure 6.7: Resolution time vs. formula length for the Guarded Fragment (GF).

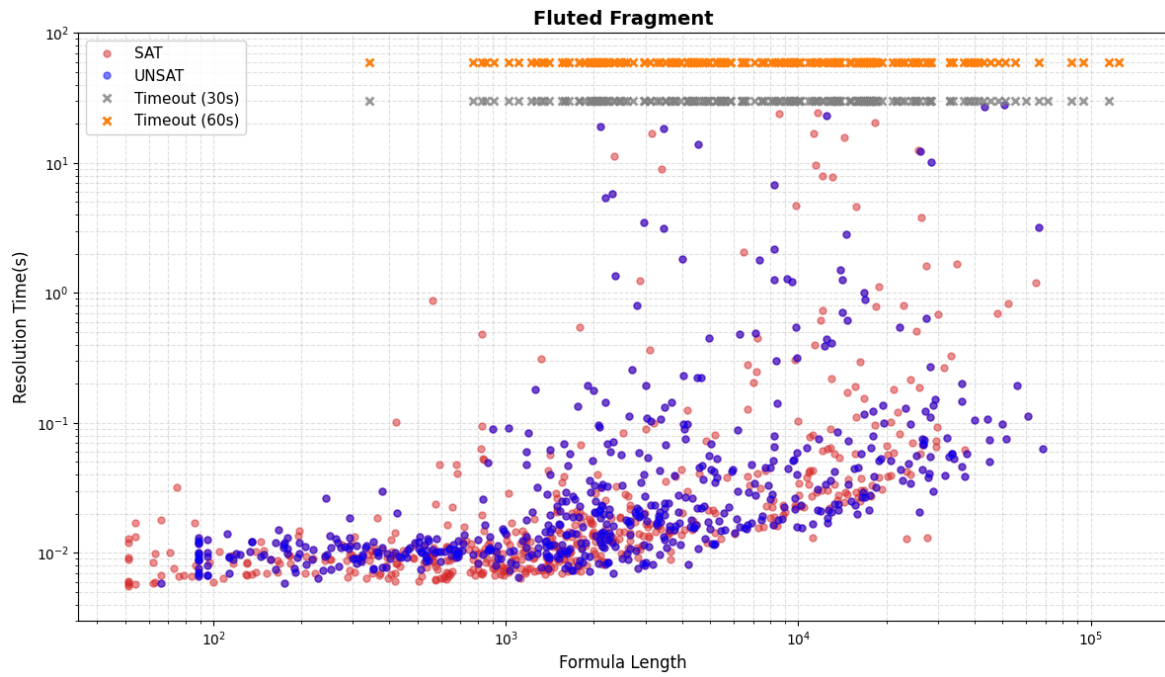


Figure 6.8: Resolution time vs. formula length for the Fluted Fragment (FL).

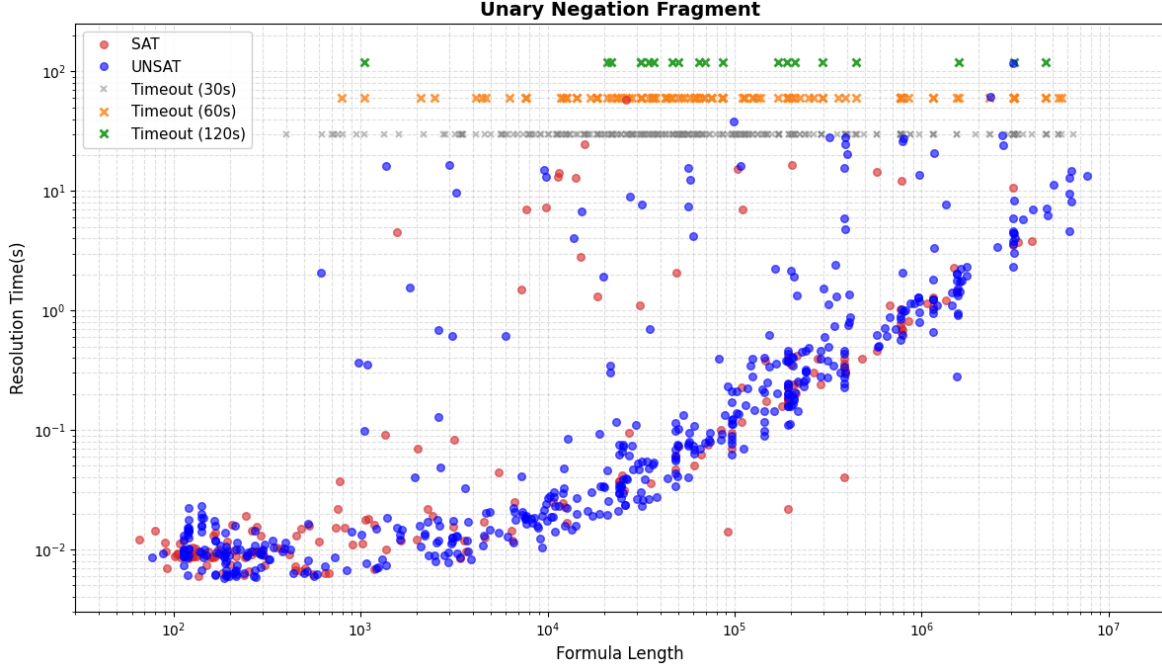


Figure 6.9: Resolution time vs. formula length for the Unary Negation Fragment (UNFO).

6.5 Future Research Directions

While this research establishes a solid foundation for the automated generation of decidable First-Order Logic formulas, it opens several distinct avenues for future exploration within automated reasoning and AI:

- **Expansion of SAT Generation:** The finite-model-driven SAT generation pipeline, currently deployed for the Two-Variable Fragment (FO^2), should be mathematically formalized and extended to the remaining fragments (GF, UNFO, FL, Modal). This would provide a complete, bidirectional stress-testing framework for theorem provers.
- **Integration of Additional Fragments:** The generator's architecture can be extended to include other prominent decidable classes, such as the Bernays-Schönfinkel-Ramsey (BSR) fragment or the Ackermann class, broadening the topological diversity of the generated datasets.
- **Advanced UNSAT Construction:** The current methodology relies on a baseline structural contradiction ($\phi \wedge \neg\phi'$). Future iterations should

investigate more sophisticated, algebraic approaches to UNSAT generation, producing deeply embedded unsatisfiable cores that challenge prover heuristics.

- **Extended Benchmarking:** The empirical evaluation phase can be scaled to include a wider array of automated theorem provers (e.g., E, SPASS, cvc5) and broader hardware configurations, aiming to discover new algorithmic bottlenecks and performance anomalies.
- **Decision Procedures in Vampire:** Ongoing academic efforts to map specific saturation strategies and CASC portfolio modes in Vampire to particular logical fragments should be continued. Tailoring internal ATP heuristics to recognize and exploit known decidable topologies could drastically reduce search spaces.
- **Automated Reasoning:** The massive, precisely labeled datasets generated by this tool present an ideal training ground for AI. Future research could leverage these datasets to train Large Language Models (LLMs) or Graph Neural Networks (GNNs) to predict satisfiability, select optimal proof strategies, or even synthesize proofs based purely on syntactic structure.

Concluding Remarks

The capacity to systematically evaluate Automated Theorem Provers hinges on the availability of diverse, structurally complex, and mathematically rigorous test data. This research bridges a critical gap in the automated reasoning ecosystem by introducing a highly scalable, fragment-aware formula generator. By enforcing strict constraints from Fluted variable sequences to Guarded quantifiers, the framework allows researchers to push state-of-the-art provers like Vampire to their computational limits in a controlled manner.

Ultimately, mapping the temporal trajectories and resolution boundaries of decidable fragments does more than just benchmark software; it provides a vital diagnostic lens. Exposing where modern heuristics excel and where they inevitably degrade forms the essential first step toward developing the next generation of context-aware reasoning engines.

List of Figures

1.1	The hierarchical parse tree representing the syntactic structure of the formula $\forall x(S(x) \rightarrow \exists y(C(y) \wedge E(x, y)))$	8
2.1	Syntactic tree of the FO^2 formula (2.1). The active variable pool is passed down the tree and strictly bounded to $\{x, y\}$. Notice how the innermost existential quantifier over x rebinds the variable (denoted as x'), shadowing the outermost universal binding.	12
2.2	Syntactic tree of the fluted formula (2.2). The active variable stack is passed down the tree, and atomic leaves deterministically consume a contiguous suffix of it.	15
2.3	Syntactic tree of the guarded formula (2.3) assuming an initial active variable x_1 . The dynamic active scope grows as quantifiers are introduced. Each quantification spawns a composite node structure where an explicit guard atom shields the sub-formula, strictly consuming the current variable scope.	17
2.4	Syntactic tree of the valid UNFO formula (2.4). The negation node acts as a strict structural filter, ensuring that the sub-tree below it depends on at most one free variable.	19
2.5	Syntactic tree of the translated modal formula (2.6). The translation strictly alternates between w_1 and w_2 . When the inner diamond operator ($\Diamond q$) is expanded, it reuses w_1 for its target world, completely shadowing the original initial world.	22
3.1	Class Diagram representing the AST architecture. The abstract base class ASTNode acts as the root, with specific final classes implementing the concrete nodes. Composition relationships highlight how nodes manage their children through exclusive ownership.	25
3.2	UML Class Diagram illustrating the Template Method pattern. FormulaBuilder defines the algorithmic skeleton, while concrete generator classes implement the abstract hooks to enforce their specific syntactic rules without duplicating the orchestration logic.	27

3.3	UML Class Diagram detailing the semantic SAT generation architecture. The F02SATGenerator orchestrates the FiniteModel . It relies on the Visitor Pattern via the F02Evaluator to safely traverse the AST and compute semantic truth values without altering the underlying node structures.	29
3.4	UML Component Diagram illustrating the integration of the Vampire Actor. The VampireRunner class acts as the structural bridge between the internal generation pipeline and the external ATP process, parsing the SZS output to validate formula satisfiability.	30
3.5	Global System Architecture. The diagram illustrates the complete data flow.	32
3.6	The compact Command Line Interface help menu, displaying the configuration options grouped by functional category.	34
5.1	Conceptual diagram of the AVATAR architecture adapted from [15]. The workload is split between a standard superposition engine and an embedded SAT solver, exchanging abstractions and models to efficiently determine satisfiability.	53
6.1	SZS status distribution across FO^2 scaling datasets.	56
6.2	SZS status distribution across Fluted logic isolated scaling datasets.	57
6.3	SZS status distribution across Guarded Fragment scaling datasets.	58
6.4	SZS status distribution across UNFO scaling datasets.	59
6.5	SZS status distribution across Modal logic scaling datasets.	60
6.6	Resolution time vs. formula length for the FO^2 fragment.	62
6.7	Resolution time vs. formula length for the Guarded Fragment (GF).	63
6.8	Resolution time vs. formula length for the Fluted Fragment (FL).	64
6.9	Resolution time vs. formula length for the Unary Negation Fragment (UNFO).	65

List of Algorithms

1	NNF Transformation for AST Nodes	37
2	Recursive AST Generation via FormulaBuilder	39
3	Fluted Fragment: Stack-Depth and Generation	40
4	Guarded Fragment: Guarded Quantifier Generation	41
5	Unary Negation: Constrained Negation Generation	41
6	Modal Logic: Standard Translation Generation	42
7	Budget-Constrained UNSAT Generation	43
8	Target Propagation for FO^2 SAT Generation	44
9	Quantifier Target Expansion (expandTargets)	45
10	$\mathcal{FL}$ Forced Quantification	47
11	GF Existential Guard Generation	48
12	UNFO Negation Scope Truncation	48

Bibliography

- [1] Hajnal Andréka, Johan van Benthem, and István Németi. Modal languages and bounded fragments of predicate logic. *Journal of Philosophical Logic*, 27(3):217–274, 1998.
- [2] Leo Bachmair and Harald Ganzinger. Rewrite-based equational theorem proving with selection and simplification. *Journal of Logic and Computation*, 4(3):217–247, 1994.
- [3] Alonzo Church. A note on the Entscheidungsproblem. *Journal of Symbolic Logic*, 1(1):40–41, 1936.
- [4] Herbert B. Enderton. *A Mathematical Introduction to Logic*. Academic Press, San Diego, CA, 2 edition, 2001.
- [5] Melvin Fitting. *First-Order Logic and Automated Theorem Proving*. Springer, New York, NY, 2 edition, 1996.
- [6] Kurt Gödel. Die vollständigkeit der axiome des logischen funktionenkalküls. *Monatshefte für Mathematik und Physik*, 37(1):349–360, 1930.
- [7] Erich Grädel. On the restraining power of guards. *Journal of Symbolic Logic*, 64(4):1719–1742, 1999.
- [8] Erich Grädel, Phokion G. Kolaitis, and Moshe Y. Vardi. On the decision problem for two-variable first-order logic. *Bulletin of Symbolic Logic*, 3(1):53–69, 1997.
- [9] Michael Huth and Mark Ryan. *Logic in Computer Science: Modelling and Reasoning about Systems*. Cambridge University Press, Cambridge, UK, 2 edition, 2004.
- [10] Laura Kovács and Andrei Voronkov. First-order theorem proving and Vampire. In *Proceedings of CAV*, volume 8044 of *LNCS*, pages 1–35, 2013.
- [11] Ian Pratt-Hartmann, Wiesław Szust, and Lidia Tendera. Quine’s fluted fragment revisited. In *Proceedings of LICS*, 2018.
- [12] Willard V. Quine. Algebraic logic and predicate functors. *The Ways of Paradox and Other Essays*, 1969.

- [13] Balder ten Cate and Luc Segoufin. Unary negation. *Logical Methods in Computer Science*, 9(3), 2013.
- [14] Alan M. Turing. On computable numbers, with an application to the Entscheidungsproblem. *Proceedings of the London Mathematical Society*, 42:230–265, 1936.
- [15] Andrei Voronkov. AVATAR: The architecture for first-order theorem provers. In *Computer Aided Verification (CAV)*, volume 8559 of *Lecture Notes in Computer Science*, pages 696–710. Springer, 2014.
- [16] Alfred North Whitehead and Bertrand Russell. *Principia Mathematica*, volume 1–3. Cambridge University Press, Cambridge, 1910–1913.